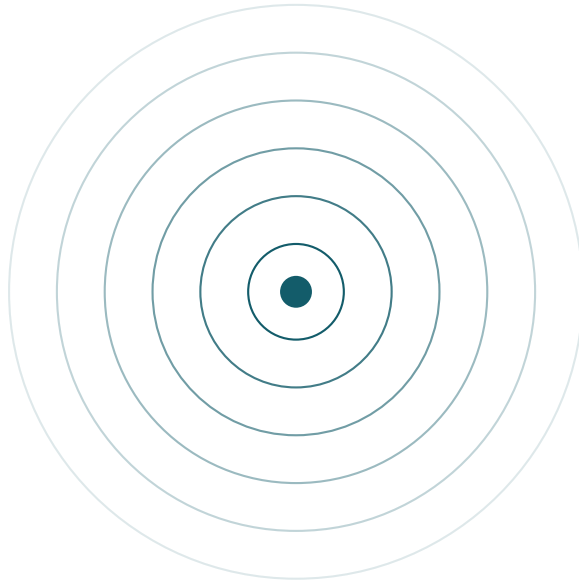


From your first command to a safe history of every change



Linux and Git from Zero

A Practical Guide

FIRST EDITION

Linux and Git from Zero

*A Practical Guide to the Terminal,
and to Never Losing Your Work*

FIRST EDITION

Linux and Git from Zero: A Practical Guide

Copyright © 2026 The Author. All rights reserved.

Revision History for the First Edition

2026-09-26 First release

The programs in this book are included for their instructional value. They have been tested with Git 2.x and the standard command-line tools on Debian 13 and macOS 26, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here change over time; check their current documentation.

The files the labs use are in the `labs/` folder that accompanies this book.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with XeLaTeX. Source: Markdown, converted with pandoc.

Contents

Preface	vii
Who This Book Is For	vii
How This Book Is Organized	vii
How to Use This Book	viii
What You Need Installed	viii
Conventions Used in This Book	viii
 Part I. Understand	 1
 1. The Machine Behind the Terminal	 2
The Terminal Is a Window, Not the Computer	2
What an Operating System Does	2
Linux: A Kernel Plus Everything Around It	3
macOS and Windows: Cousins and Guests	4
The Unix Idea: Small Tools	4
Everything Is a File	4
Lab: First Look Around	5
Summary, Key Terms, and Review Questions	5
 2. The Shell: Talking to Your Computer in Words	 7
The Shell's Loop	7
The Parts of a Command	7
Where You Are: Paths and the Working Directory	8
Making, Copying, and Removing	9
How the Shell Finds a Command	9
Streams, Redirection, and Pipes	10
Getting Help	11
Lab: A Sandbox Tour	12
Summary, Key Terms, and Review Questions	12
 3. Files, Folders, and Who May Touch Them	 14
One Tree from /	14
Hidden Files	15

Users and Groups	15
Reading Permissions	16
Changing Permissions with <code>chmod</code>	17
Three Reasons for “Permission Denied”	17
<code>sudo</code> : Borrowing Root for One Command	18
Least Privilege	18
Lab: Lock It Down	19
Summary, Key Terms, and Review Questions	19
4. Why Code Needs a History	21
Version Control: A Photo Album for Your Project	21
Commits: Snapshots with a Caption	21
Branches: Bookmarks in the Album	22
Merging: Bringing Work Back Together	23
Remotes: Copies of the Project Elsewhere	23
What Git Is, and Why GitHub Is Not Git	24
Lab: Meet Git	24
Summary, Key Terms, and Review Questions	25
Part II. Use	27
5. The Everyday Command Line	28
Finding Files	28
Viewing Files	29
Searching Inside Files with <code>grep</code>	30
Changing Text with <code>sed</code>	31
Sorting and Counting	31
Building a Pipeline	31
JSON with <code>jq</code>	32
Environment Variables	33
Lab: Interrogate a Log	33
Summary, Key Terms, and Review Questions	34
6. Programs That Keep Running	35
Processes: Programs in Motion	35
Foreground and Background	35
Signals: Asking a Process to Stop	36
Services: Programs the System Looks After	37
Logs: Where the Output Goes	38
Scheduled Jobs with <code>cron</code>	39
Lab: A Program That Outlives the Terminal	39
Summary, Key Terms, and Review Questions	40
7. Installing Software and Working Remotely	41
Package Managers	41

Virtual Environments for Python	42
SSH: A Terminal on Another Machine	42
Copying Files Between Machines	43
Keeping Work Alive with tmux	44
Lab: Your Toolbox	44
Summary, Key Terms, and Review Questions	44
8. Git Every Day.	46
Starting a Repository.	46
Three Places Your Work Lives	47
The Everyday Loop: Add and Commit.	47
Seeing What Changed.	48
Keeping Files Out with .gitignore	49
Writing Good Commit Messages	50
Branches and Merging	50
Push and Pull	52
Undoing Mistakes.	53
Lab: Save Your Work Properly	54
Summary, Key Terms, and Review Questions	55
Part III. Apply	56
9. Lab: Automate a Daily Chore.	57
Step 1: Do the Chore by Hand	57
Step 2: Put the Commands in a File	58
Step 3: Watch It Fail Badly.	59
Step 4: Add the Safety Line	59
Step 5: Use Variables, and Quote Them	60
Step 6: Make Decisions with if	60
Step 7: Repeat with a Loop	61
Step 8: Accept an Argument and Fail Clearly.	61
Step 9: Let shellcheck Review It.	63
Step 10: Schedule It	64
Summary, Key Terms, and Review Questions	64
10. Lab: Try an Idea Without Breaking Anything	66
Step 1: A Project That Works	66
Step 2: Try the Idea on a Branch	67
Step 3: Meanwhile, main Moves On	67
Step 4: Merge, and Meet a Conflict	68
Step 5: Resolve It	68
Step 6: Rebasing, in Brief.	69
Step 7: Lose a Commit, Then Find It	70
Summary, Key Terms, and Review Questions	71

11. Lab: Work on a Project with Someone Else.	73
Step 1: Set Up the Shared Remote	73
Step 2: Both Work at Once.	74
Step 3: Fetch, Look, Then Pull	75
Step 4: A Feature Branch	76
Step 5: The Pull Request and the Review	76
Step 6: Merge and Clean Up	77
Step 7: Forks	78
The Open-Source Workflow	78
Summary, Key Terms, and Review Questions	79
 Part IV. What Next	 81
12. Security, Speed, Failure, and Cost	82
Security: What Can Go Wrong, and What Limits the Damage	82
Speed: What Makes This Slow	83
Failure: How It Breaks, and How to Diagnose It	84
Cost: What This Costs You	85
13. Where to Go from Here	86
Make It a Habit First	86
What to Learn Next	86
What to Skip for Now	87
One Last Thing	87
 Appendices	 88
A. Command Cheat Sheet	88
Files, Folders, and Text	88
The Shell, Permissions, and Settings	89
Processes, Software, and Remote Work	89
Scripts	89
B. Git Cheat Sheet	90
Set Up Once	90
Every Day	91
Sharing	91
Undo	92
Resolving a Conflict	92
C. Glossary	93
Index	96

Preface

Every tutorial says “open the terminal”, and I don’t know what I’m typing. When my code breaks, I can’t get the old version back.

If that sounds like you, this book is for you. It answers one question: *how do I use the tools every developer uses every day, and never lose my work?*

The tools are the command line (the shell, on Linux, macOS, or Windows with WSL) and Git. They are not glamorous. But nearly every other skill in software sits on top of them, and once they stop feeling like magic spells, everything else gets easier.

Who This Book Is For

You know basic Python: variables, functions, loops, lists, and dictionaries, and you have run a `.py` file. You do not need to know anything about Linux, the terminal, or Git. If you have copied commands from tutorials without knowing what they do, this book is written for you. Everything is explained when it’s needed, in the order you need it.

How This Book Is Organized

- *Part I, Understand*, gives you the mental model. What the terminal is, what the shell does with the words you type, how files and permissions work, and why code needs a history. There is very little to type yet.
- *Part II, Use*, makes you a confident everyday user: finding and searching files, keeping programs running, installing software, working on remote machines, and the Git commands you’ll use every single day.
- *Part III, Apply*, is three guided labs. You automate a chore with a script, try a risky idea on a Git branch, and work on a shared project with a second person.
- *Part IV, What Next*, looks at security, speed, failure, and cost for these tools, then tells you what to learn next and what to skip for now.
- *The appendices* hold a command cheat sheet, a Git cheat sheet, and a glossary.

How to Use This Book

Read Part I in order. It is short, and the rest of the book leans on it. After that, keep a terminal open next to the book and type the commands yourself. Typing is slower than pasting, and that is the point: your fingers learn what your eyes skim.

Every chapter in Parts I and II ends with a lab, and the chapters of Part III are labs. Each lab step says what you should see, so you can check your work. Do the labs in a throwaway folder such as `~/sandbox`, never inside a project you care about. The sample files the labs use are in the book's lab files: download *Linux and Git from Zero Lab Files.zip* from this book's entry on the Towards Intelligence CS & Engineering on-ramp (robotbrainlab.github.io/cse/on-ramp.html) and unzip it. It unpacks to a folder called `labs/`.

When you meet a word you don't know, look it up in Appendix C (Glossary). Every bold term in the book is defined there in a line.

What You Need Installed

- *A terminal.* Linux and macOS have one built in. On Windows, install WSL (Windows Subsystem for Linux) with `wsl --install` in PowerShell, then use the Ubuntu terminal it gives you. Chapter 1 explains what WSL is.
- *Git 2.28 or newer* (check with `git --version`). On macOS, running `git` the first time offers to install it. On Ubuntu or WSL, run `sudo apt install git`.
- *Python 3.10 or newer* (check with `python3 --version`).
- *jq* and *shellcheck*, two small tools used in Chapters 5 and 9. Install them with `sudo apt install jq shellcheck` on Ubuntu or WSL, or `brew install jq shellcheck` on macOS.
- *A text editor.* VS Code is the usual choice, but any editor that saves plain text will do.

You do not need a GitHub account or a server. The labs use a shared folder on your own computer to stand in for GitHub.

The examples were tested with Git 2.52, bash 5.2 on Debian 13, zsh on macOS 26, Python 3.14, jq 1.8, and ShellCheck 0.11.

Conventions Used in This Book

- Bold type marks a new term the first time it is explained. Every bold term is also in Appendix C.

- Constant width marks commands, file names, and anything else you would type exactly.
- Code blocks show commands, output, or Python. Commands are shown without a prompt, so you can paste them as they are. When a block mixes commands and their results, the result follows in its own block.
- Output was copied from real runs. Your user name, dates, sizes, and Git commit IDs will differ. The example user is `ada`, with a home folder of `/home/ada`. On macOS your home folder is `/Users/<you>` instead.
- A line containing only `...` means some output was left out.
- Where Linux and macOS behave differently, the text shows both.
- Three kinds of callout appear in the text:



A suggestion that saves you time or trouble.



Something to be aware of, or a detail that explains what you're seeing.



A mistake that can cost you data, money, or security. Read these twice.

PART I

Understand



What to ignore for now: how the kernel manages memory, the history of Unix, and Git's internal file formats.

The Machine Behind the Terminal

Before you type a single command, you need a picture of what you’re talking to. This chapter gives you that picture: what the terminal window is, what sits behind it, what Linux really is, and the two simple ideas that explain why the command line works the way it does.

The problem. Every tutorial starts with “open a terminal”, but I don’t know what that window actually is.

The question. What is that window, and what is on the other side of it?

The Terminal Is a Window, Not the Computer

The **terminal** is just a window that shows text and sends your keystrokes somewhere. On its own it does nothing. It’s like a phone: useful only because someone answers at the other end.

The one who answers is the **shell**: a program that reads the line you type, runs the right program, shows you the result, and waits for the next line. Chapter 2 is all about the shell. The programs it runs, in turn, ask the operating system for everything they need.

What an Operating System Does

An **operating system** is the software that shares one computer among many programs. Your laptop may be running a browser, an editor, and fifty background helpers at once. Something has to decide who runs next, whose memory is whose, and who may open which file.

The heart of the operating system is the **kernel**. Think of a block of flats. The tenants are your programs. The kernel is the building manager. Tenants can’t walk into the boiler room or

another tenant’s flat. When they need something, they ask the manager, and the manager decides whether to allow it.

That “asking” has a name. A **system call** is a request from a program to the kernel: “open this file”, “start this program”, “send these bytes”. Programs never touch the disk or the network directly. They make system calls, and the kernel does the work, after checking that they’re allowed.

Everything outside the kernel is called userspace. Your shell, `ls`, Python, your editor, and your own scripts all live there.

USERSPACE: shell, <code>ls</code> , python, your programs
system calls: the only door into the kernel
KERNEL: files, processes, memory, devices
HARDWARE: disk, memory, CPU, network card

You only need to know the door is there. When a command says “Permission denied”, it is the kernel saying no at that door.

Linux: A Kernel Plus Everything Around It

Strictly speaking, **Linux** is just a kernel. On its own it gives you no shell, no `ls`, and no way to install software. To get a working system, people bundle the kernel with a shell, hundreds of small tools, a way to install more software, and sometimes a desktop.

That bundle is a **distribution**, or “distro”. The distributions you’re most likely to meet:

Distribution	Where you’ll see it
Ubuntu	Laptops, cloud servers, and WSL on Windows
Debian	Servers; Ubuntu is built on it
Fedora, Red Hat Enterprise Linux	Company servers

They share the same kernel and core ideas; mostly, they differ in how you install software (Chapter 7). What you learn on one works on all of them. And Linux runs most of the world’s servers, so the machine your code ends up on is probably Linux.

macOS and Windows: Cousins and Guests

Unix is the older operating system that Linux was modelled on. Linux copied its ideas and commands, but not its code. macOS is a direct descendant: underneath the Mac’s desktop is a real Unix system. So the terminal on a Mac speaks almost the same language as Linux. The commands are the same; a few options differ.

The differences come from two families of tools: Linux ships the GNU versions of `ls`, `sed`, `grep`, and friends, and macOS ships the BSD versions. When it matters, this book shows both.

Windows is not Unix, and its own command line, PowerShell, uses different commands. But Windows can run a real Linux inside it with **WSL**, the Windows Subsystem for Linux. For this book, a WSL terminal is a Linux terminal.

The Unix Idea: Small Tools

Unix was built on a few simple ideas that still explain almost everything you’ll see. The first is: do one thing well. `ls` lists files. `sort` sorts lines. `wc` counts lines and words. None tries to do everything.

The power comes from connecting them. The output of one tool can flow straight into the next, like water through pipes. There is no “count files” command; you join the tool that lists to the tool that counts. Chapter 2 shows you how.

A second idea surprises beginners: silence means success. A command that works often prints nothing at all. If you see output, read it; it may be an error.

Everything Is a File

The other big Unix idea is that almost everything is shown to you as a file. Your documents are files, of course. But so are folders (a folder is a file that lists other files). So are devices: on Linux, your disk appears as a file under `/dev`, and so does the terminal itself.

Here is a strange but useful one. `/dev/null` is a file that throws away whatever you write into it and is always empty when you read it:

```
echo "into the void" > /dev/null
cat /dev/null
```

Both commands print nothing. The first wrote a line into `/dev/null`, which discarded it. The second read `/dev/null` and found it empty. It’s the command line’s rubbish bin, and you’ll use it to hide output you don’t care about.

On Linux, the kernel even shows its own live state as files under `/proc`. You can read how much memory the machine has with the same tool you'd use to read a text file:

```
head -1 /proc/meminfo

MemTotal:      8024788 kB
```

So a handful of tools that read files can inspect almost anything on the system. Learn to handle files, and you can handle the whole machine.

Lab: First Look Around

Open your terminal (on Windows, the Ubuntu app from WSL). Type each command and press Enter.

1. Ask who you are: `whoami`. **Expected:** your user name, such as `ada`.
2. Ask which kernel is running: `uname -s`. **Expected:** Linux on Linux and WSL, Darwin on macOS (Darwin is the name of macOS's Unix core).
3. Find out which distribution you have with `head -2 /etc/os-release`, or on macOS, `sw_vers`. **Expected:** your system's name and version, such as:

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
```

4. Find out which shell answers you: `echo $SHELL`. **Expected:** `/bin/bash` on most Linux systems, `/bin/zsh` on macOS.
5. Look at `/dev/null` itself: `ls -l /dev/null`.

```
crw-rw-rw- 1 root root 1, 3 Sep 26 14:25 /dev/null
```

Expected: a line starting with `c`. That letter means “character device”: a file that is really a device. You'll learn to read the rest of this line in Chapter 3.

Summary, Key Terms, and Review Questions

Summary

- The terminal is only a window. The shell behind it reads your commands and runs programs.

- The kernel is the core of the operating system. Programs ask it for everything through system calls, and it can say no.
- Linux is a kernel. A distribution bundles it with a shell, tools, and a way to install software.
- macOS is a Unix system with slightly different (BSD) tools; WSL runs real Linux inside Windows.
- Unix tools are small and are combined to do big jobs. Silence usually means success.
- Almost everything, including devices, is presented as a file.

Key Terms

Term	Meaning
Terminal	The window that shows text and passes your typing to the shell
Shell	The program that reads your commands and runs them
Operating system	Software that shares one computer among many programs
Kernel	The core of the operating system; controls hardware and access
System call	A program's request to the kernel
Linux	A Unix-like kernel, and by extension the systems built on it
Distribution	The Linux kernel bundled with tools and a package manager
Unix	The older operating system whose design Linux and macOS follow
WSL	Windows Subsystem for Linux: real Linux running inside Windows

Review Questions

1. What is the difference between the terminal and the shell?
2. What does the kernel do, and why can't a program read the disk directly?
3. Why is "Linux" not, strictly, a complete operating system? What turns it into one?
4. Why do most commands you learn on Linux also work on macOS?
5. What does "silence means success" mean, and why does it matter when you read output?

You understand this when you can explain, in your own words, what happens between typing in the terminal window and the kernel doing the work.

The Shell: Talking to Your Computer in Words

This chapter turns the shell from a mystery into a tool. You'll learn to read any command, move around, see how the shell finds programs, and join small commands into bigger ones. By the end, you'll predict what a command does before you press Enter.

The problem. I type a command and something happens, but I couldn't tell you what.

The question. What exactly happens between pressing Enter and seeing the result?

The Shell's Loop

The shell does the same four things, forever:

1. It shows a prompt, the text before your cursor (often your name, the folder you're in, and a \$ or %).
2. It reads the line you type when you press Enter.
3. It finds the program you named and runs it, passing along the rest of the line.
4. It shows the program's output, then goes back to step 1.

The two shells you'll meet are bash, the default on most Linux systems, and zsh, the default on macOS. For everything in this book they behave the same way.

The Parts of a Command

Every command line has the same shape: a command, then options, then arguments.

```
ls  -l  -a  /home/ada/sandbox
```

command: which program to run

options: how to do it

argument: what to act on

- The **command** is the program to run. Here it's `ls`, which lists files.
- An **option** (also called a flag) changes how the command behaves. Short options are one letter after a dash (`-l`), and can be combined (`-la` means `-l -a`). Long options are words after two dashes (`--all`).
- An **argument** is the thing the command acts on, usually a file or folder. Most commands have a sensible default when you leave it out.

Spaces separate the parts. That's why file names with spaces cause trouble: `my notes.txt` looks like two arguments. Wrap such names in quotes: `"my notes.txt"`.

Where You Are: Paths and the Working Directory

The shell always has a current folder, called the **working directory**. Commands act there unless you say otherwise. When you open a terminal, you start in your **home directory**, your personal folder: `/home/ada` on Linux, `/Users/ada` on macOS. The shell lets you write it as `~`.

A **path** is the address of a file or folder. An absolute path starts at the very top of the file system, `/`, and works from anywhere: `/home/ada/sandbox/todo.txt`. A relative path starts from where you are now: if you're in `/home/ada`, then `sandbox/todo.txt` means the same file. Two special names help: `.` means “this folder” and `..` means “the folder above”.

Three commands do most of the moving around. `pwd` prints the working directory. `ls` lists what's in it. `cd` changes it.

```
pwd
mkdir -p sandbox/notes
cd sandbox
touch todo.txt ideas.txt
ls

/home/ada
ideas.txt  notes  todo.txt
```

`mkdir -p` made the folder `sandbox` and, inside it, `notes` (the `-p` means “make parent folders too, and don’t complain if they exist”). `touch` created two empty files. Now the long listing:

```
ls -l

total 4
-rw-r--r-- 1 ada ada    0 Sep 26 14:27 ideas.txt
drwxr-xr-x 2 ada ada 4096 Sep 26 14:27 notes
-rw-r--r-- 1 ada ada    0 Sep 26 14:27 todo.txt
```

Each line shows permissions, owner, size in bytes, date, and name. A `d` at the start marks a directory. Chapter 3 reads the rest.

To move around, `cd notes` goes into a folder, `cd ..` climbs one level, and `cd` on its own always takes you home.

Making, Copying, and Removing

A few more commands handle everyday file work:

Command	What it does
<code>mkdir -p a/b</code>	Make folders, including parents
<code>touch f</code>	Create an empty file (or update its time)
<code>cp src dst</code>	Copy a file; <code>cp -r</code> copies a folder
<code>mv old new</code>	Move or rename
<code>rm f</code>	Remove a file; <code>rm -r</code> removes a folder
<code>cat f</code>	Print a whole file



`rm` does not use the bin or the Trash. The file is gone the moment you press Enter. Be especially careful with `rm -r`, and read the whole line before you press Enter.

How the Shell Finds a Command

When you type `ls`, how does the shell know which program you mean? It looks in a list of folders stored in a setting called **PATH**. It checks them from left to right and runs the first program with that name.

```
echo $PATH
```

```
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

The folders are separated by colons, which tells you where a command was found:

```
which ls python3
```

```
/usr/bin/ls
```

```
/usr/local/bin/python3
```

A few commands, like `cd`, aren't programs at all. They're a builtin, part of the shell itself:

```
type cd
```

```
cd is a shell builtin
```

This explains the most common error in the terminal:

```
bash: nosuchcmd: command not found
```

No folder on your `PATH` holds a program with that name. It's misspelled, not installed, or installed in a folder that isn't on your `PATH`.

Streams, Redirection, and Pipes

Every program starts with three streams, like three hoses attached to it. **Standard input** (`stdin`) is where it reads from, usually your keyboard. **Standard output** (`stdout`) is where it writes results, usually your screen. **Standard error** (`stderr`) is a second output just for error messages, also your screen.

Redirection points those hoses somewhere else. `>` sends `stdout` into a file, replacing what was there. `>>` adds to the end instead. `2>` sends `stderr` into a file.

```
cd ~/sandbox
echo "buy milk" > todo.txt
echo "call Sam" >> todo.txt
cat todo.txt
```

```
buy milk
call Sam
```

Errors travel on their own hose, so you can separate them:

```
ls /nope
ls /nope 2> errors.txt
cat errors.txt
```

```
ls: cannot access '/nope': No such file or directory
ls: cannot access '/nope': No such file or directory
```

The first `ls` printed its complaint on the screen. The second printed nothing, because its error went into `errors.txt`. On macOS the message reads `ls: /nope: No such file or directory`; the idea is the same.

A **pipe**, written `|`, connects one program's stdout to the next program's stdin. This is how small tools join forces. How many entries are in `/etc`? `ls` lists them, `wc -l` counts lines:

```
ls /etc | wc -l

75
```

Read a pipeline left to right, like a sentence: “list `/etc`, then count the lines”. Chapter 5 builds longer ones.

Getting Help

You never need to memorise options. Every standard command has a manual page, or **man page**. Run `man ls` to read it; use the arrow keys or space to scroll, `/word` to search, and `q` to quit. Most GNU tools on Linux also print a short summary with `--help`:

```
ls --help | head -1
```

Usage: `ls [OPTION]... [FILE]...`

macOS's `ls` doesn't understand `--help`; use `man ls` there.



Press Tab while typing a name, and the shell completes it for you; press it twice to see the choices. It saves typing and, more importantly, typos.

A few more keys make life easier. The Up arrow brings back earlier commands. Ctrl-R searches your history as you type. Ctrl-C cancels whatever is running. Ctrl-L clears the screen.

Lab: A Sandbox Tour

Work in a new terminal window. Each step says what you should see.

1. Make a fresh folder for this lab: `mkdir -p ~/sandbox/ch2/notes`, then `cd ~/sandbox/ch2`. **Expected:** `pwd` prints your home folder followed by `/sandbox/ch2`.
2. Create a file with two lines: `echo "first" > log.txt`, then `echo "second" >> log.txt`, then `cat log.txt`. **Expected:** first and second on two lines.
3. Replace instead of adding: `echo "only" > log.txt` and `cat log.txt`. **Expected:** just only. That's the difference between `>` and `>>`.
4. Copy and move: `cp log.txt notes/copy.txt`, then `mv notes/copy.txt notes/moved.txt`, then `ls notes`. **Expected:** `moved.txt`.
5. Separate an error: `ls /nope 2> err.txt`, then `cat err.txt`. **Expected:** nothing on screen from the first command; the error message from the second.
6. Count with a pipe: `ls ~/sandbox/ch2 | wc -l`. **Expected:** 3 (`err.txt`, `log.txt`, `notes`).
7. Find where Python lives: `which python3`. **Expected:** a path such as `/usr/bin/python3`.
8. Look up an option: run `man wc` and find what `-w` does. **Expected:** it counts words. Press q to leave.

Summary, Key Terms, and Review Questions

Summary

- The shell loops: prompt, read, run, show. `bash` and `zsh` work the same way for everyday use.
- A command line is a command, then options, then arguments, separated by spaces.

- The working directory is where commands act. Absolute paths start at /; relative paths start where you are.
- The shell finds programs by searching the folders on your PATH, left to right.
- Every program has stdin, stdout, and stderr. Redirection points them at files; pipes connect programs.
- `man` and `--help` explain any command.

Key Terms

Term	Meaning
Option	A setting that changes how a command behaves, like <code>-l</code>
Argument	What a command acts on, usually a file or folder
Working directory	The folder the shell is currently in
Home directory	Your personal folder, written <code>~</code>
Path	The address of a file or folder
PATH	The list of folders the shell searches for commands
Standard input, output, error	The three streams every program starts with
Redirection	Sending a stream to or from a file (<code>></code> , <code>>></code> , <code>2></code>)
Pipe	The <code>'</code>
Man page	The built-in manual for a command

Review Questions

1. In `cp -r photos /home/ada/backup`, which part is the command, which the option, and which the arguments?
2. You're in `/home/ada/sandbox`. Write the absolute and relative paths to `notes/ideas.txt`.
3. What's the difference between `>` and `>>`? Which one can destroy data?
4. What does "command not found" tell you, and what are three possible causes?
5. What does `ls /etc | wc -l` do, step by step?

You understand this when you can read a command like `ls -l ~/sandbox 2> err.txt | wc -l` aloud and say what each part does.

Files, Folders, and Who May Touch Them

This chapter gives you a map and a set of keys. The map is the single tree that every file on a Unix system hangs from, so you know where things live. The keys are users, groups, and permissions, so you know who may read, change, or run each file, and how to fix “Permission denied” without making things worse.

The problem. It says “Permission denied”, and I don’t know whose permission I need.

The question. Who decides what I’m allowed to do with a file, and how do I get permission the right way?

One Tree from /

Windows gives each drive a letter: C:, D:. Unix doesn’t. Everything hangs from one tree, and the top of that tree is the root directory, written /. Every disk, every USB stick, every device appears somewhere under it.

Programs follow a shared convention about where things go, so once you know the map, you can guess where to look:

Path	What lives there
/home/ada	Your home folder (on macOS, /Users/ada)
/root	The home folder of the administrator account
/etc	System-wide settings, as plain text files
/bin, /usr/bin	Programs: ls, cat, python3
/usr/local/bin, /opt	Programs you installed yourself
/var/log	Log files that programs write as they run
/tmp	Temporary files, often wiped at restart
/dev	Devices, shown as files
/proc	Live information from the kernel (Linux only)

macOS adds its own folders (/Applications, /Users, /Library), and Homebrew installs programs under /opt/homebrew on Apple silicon Macs. The idea is the same.

Hidden Files

A file whose name starts with a dot is a **hidden file**, often called a dotfile. Plain `ls` skips them. `ls -a` shows everything:

```
ls -a
.  ..  .bash_logout  .bashrc  .profile  sandbox
```

Dotfiles are mostly settings. `.bashrc` configures bash each time it starts; `.zshrc` does the same for zsh. Later you'll meet `.ssh` (your keys), `.gitconfig`, and `.gitignore`. They're hidden to keep them out of your way, not to keep them secret.

Users and Groups

Unix was built for many people sharing one machine, so every file and every running program belongs to someone. A user is an account: a name, a numeric UID (user ID), and a home folder. A group is a named set of users, handy for sharing files with a team.

Changing Permissions with chmod

chmod (change mode) changes permissions. The readable form names who (u user/owner, g group, o others, a all), then + to add or - to remove, then the letters:

```
chmod u+x hello.sh
chmod go-r secret.txt
```

The first lets you run `hello.sh`. The second stops group members and others from reading `secret.txt`.

The short form uses numbers. Each permission has a value (read 4, write 2, execute 1), and you add them up for each class, in the order owner, group, others:

Number	Letters	Common use
600	rw-----	Secrets: only you can read or write
644	rw-r--r--	Normal files: you write, everyone reads
700	rw-x-----	Private folders and personal scripts
755	rw-xr-xr-x	Programs and shared folders

```
chmod 600 secret.txt
ls -l secret.txt
```

```
-rw----- 1 ada ada 15 Sep 26 14:31 secret.txt
```

To change who owns a file, use `chown`, as in `sudo chown ada:ada report.txt`. Giving files away is an administrator's job, so it needs `sudo`, which is explained below.

Three Reasons for “Permission Denied”

Almost every “Permission denied” you’ll meet has one of three causes.

It isn't yours. Files under `/etc` belong to root. You may read most of them, but not change them, and a few you may not even read:

```
ls -l /etc/shadow
cat /etc/shadow

-rw-r----- 1 root shadow 524 Sep 26 14:31 /etc/shadow
cat: /etc/shadow: Permission denied
```

It isn't executable. A new file never has the `x` permission. To run a script as a program, you must add it:

```
./hello.sh
chmod u+x hello.sh
./hello.sh

bash: ./hello.sh: Permission denied
Hello from a script
```

You can't get through the folder. To reach a file, you need `x` on every folder along its path. Here, a second user, ben, tries to look into Ada's home folder, which is `drwx-----`:

```
ls: cannot open directory '/home/ada': Permission denied
```

Each time, `ls -l` on the file and its folders shows you which rule said no.

sudo: Borrowing Root for One Command

Sometimes you really do need to change a system file or install software for the whole machine. For that, you use **sudo** (“superuser do”). It runs one command as root, after asking for *your* password, and records that you did it.

```
sudo apt install tree

[sudo] password for ada:
```

Type your password (nothing appears as you type) and press Enter. On macOS the prompt is just `Password:`. Only users an administrator has allowed may use `sudo`; on your own laptop, that's you.



`sudo` switches off every safety check. Use it only when a command must change the system. Never use it to fix “Permission denied” on files in your own home folder; that usually means something was created with `sudo` earlier and now belongs to root. And never “fix” errors with `chmod 777`, which lets every user and every program change the file.

Least Privilege

The rule behind all of this is **least privilege**: give each person and each program exactly the access it needs, and no more. Your notes don't need to be readable by others, so make them

600. A script needs `x` only for you, so use `u+x`, not `777`. A web app needs to read its code and write its data folder; it doesn't need root.

Least privilege limits the damage when something goes wrong. A buggy script running as you can hurt your files. The same script running as root can wreck the whole machine.

Lab: Lock It Down

Make a fresh folder for this lab and work there: `mkdir -p ~/sandbox/ch3`, then `cd ~/sandbox/ch3`.

1. List your home folder with hidden files: `ls -a ~`. **Expected:** entries starting with a dot, such as `.bashrc` (Linux) or `.zshrc` (macOS).
2. Check who you are: `id`. **Expected:** your name, your UID, and your groups. On macOS your UID is usually 501 and your groups include `staff` and `admin`.
3. Create a secret: `echo "API_KEY=abc123" > secret.txt`, then `ls -l secret.txt`. **Expected:** `-rw-r--r--`, readable by everyone.
4. Lock it: `chmod 600 secret.txt`, then `ls -l secret.txt`. **Expected:** `-rw-----`.
5. Make a script: `echo 'echo "Hello from a script"' > hello.sh`, then run `./hello.sh`. **Expected:** Permission denied.
6. Make it runnable: `chmod u+x hello.sh`, then `./hello.sh`. **Expected:** Hello from a script, and `ls -l hello.sh` shows `-rwxr--r--`.
7. Block a folder: `mkdir box`, `chmod u-x box`, then `cd box`. **Expected:** a “Permission denied” error (bash and zsh word it slightly differently). Restore it with `chmod u+x box`.
8. Try a system file: `touch /etc/myapp.conf`. **Expected:** Permission denied. Don't fix it; it's supposed to fail.

Summary, Key Terms, and Review Questions

Summary

- Everything hangs from one tree rooted at `/`; settings live in `/etc`, programs in `/usr/bin`, logs in `/var/log`, your files in your home folder.
- Names starting with a dot are hidden; `ls -a` shows them.
- Every file has an owner, a group, and `rw`x permissions for owner, group, and others. On folders, `x` means “may enter”.
- `chmod` changes permissions, in letters (`u+x`) or numbers (`600`, `644`, `755`).
- “Permission denied” means it isn't yours, it isn't executable, or a folder on the way is closed.

- `sudo` runs one command as root. Use it rarely, and follow least privilege.

Key Terms

Term	Meaning
Hidden file (dotfile)	A file whose name starts with <code>.</code> , usually settings
root	The superuser (UID 0), who bypasses permission checks
Permissions	The <code>rwx</code> rules for owner, group, and others
sudo	Runs one command as root after checking your password
Least privilege	Give each user or program only the access it needs

Review Questions

1. Where would you look for a program's system-wide settings? For its log files?
2. Read `-rw-r-----` aloud: who can do what?
3. What number would you give `chmod` for a file only you may read and write? For a script everyone may run?
4. Why do you need `x` on a folder to open a file inside it?
5. Why is "just use `sudo`" a poor fix for a permission error in your own home folder?

You understand this when you can look at any "Permission denied", run `ls -l` on the file and its folders, and say which rule refused you.

Why Code Needs a History

This chapter explains version control before you type a single Git command. You'll learn to see a project as a series of snapshots, what commits, branches, merges, and remotes are, and how Git differs from GitHub. With this picture in your head, the commands in Chapter 8 will make sense instead of feeling like spells.

The problem. My code worked yesterday. I changed something, and now I can't get yesterday back.

The question. How can I keep every working version of my project, and move between them safely?

Version Control: A Photo Album for Your Project

Version control is a system that records the state of your project at moments you choose, so you can look back at any of them, compare them, and return to them. Think of a photo album. Every time the project reaches a state worth keeping, you take a photo of the whole thing and put it in the album with a caption. Later you can flip back to any page.

Each photo is a snapshot: the full contents of every file in the project at that moment. You don't photograph just the part you changed; the snapshot is the whole project. (Behind the scenes Git stores unchanged files only once, so this costs very little space.)

The album itself, with all its snapshots, is the **repository**, or repo. It lives in a hidden folder inside your project, so the project carries its own history with it.

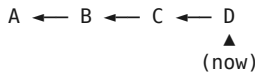
Commits: Snapshots with a Caption

A **commit** is one snapshot plus a few facts about it:

- *Who* made it (a name and email).
- *When* it was made.

- *Why*: a short message you write, like “Add login form”.
- *What came before*: a link to the previous commit, called its parent.

Those parent links chain the commits together into a history. Each commit points back to the one before it:



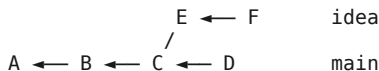
Every commit gets a unique ID, a long string of letters and digits computed from its contents, such as `b7e4f09c1d...`. People usually use just the first seven characters. The ID is how you name a commit when you want to look at it or go back to it.

A commit is deliberate. You decide when your work is in a state worth saving, and you describe it. That’s different from autosave: the history records meaningful steps, not every keystroke. A good rule is to commit whenever something small works: a function is written, a bug is fixed, a test passes.

Branches: Bookmarks in the Album

What if you want to try a risky idea without disturbing the version that works? You start a new line of commits that splits off from the main one. That separate line is a **branch**.

Here’s the surprise: a branch isn’t a copy of your files. It’s a bookmark. A branch is just a name that points at one commit. When you make a new commit on a branch, the bookmark moves forward to it.



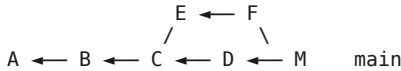
Here `main` is the usual name for the main line, and `idea` is a branch that split off at C. Commits E and F exist only on `idea`; `main` doesn’t see them. You can switch between branches at any time, and Git swaps the files in your folder to match.

How does Git know which branch you’re on? It keeps one more marker, **HEAD**, which means “you are here”. **HEAD** normally points at a branch, and the branch points at a commit.

Because branches are just bookmarks, they’re cheap. Making one is instant, so developers make them for every feature, every fix, and every experiment.

Merging: Bringing Work Back Together

If the idea works, you want it in `main`. A **merge** combines the work of two branches. Git finds the commit where they split (C), works out what each side changed since then, and applies both sets of changes. The result is a new commit with two parents:



Most merges are automatic. If one side changed `login.py` and the other changed `README.md`, there's nothing to decide. But if both sides changed the same lines of the same file in different ways, Git can't know which one you want. That's a **merge conflict**: Git stops, marks the disputed lines in the file, and asks you to choose. It sounds scary, and it's routine. Chapter 10 walks you through one.

If the idea fails, you simply switch back to `main` and delete the branch. `main` never knew about it.

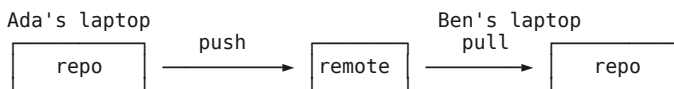
Remotes: Copies of the Project Elsewhere

So far the album sits on your laptop. If your laptop dies, the history dies with it. And a colleague can't add their own photos.

The fix is to keep another copy of the repository somewhere else, usually on a server. That other copy is a **remote**. Unlike older systems, Git doesn't treat the server as special. Every copy is a complete repository with the full history. Making your own full copy of an existing repository is called cloning, and the copy is a **clone**.

Copies are kept in step with two actions:

- **Push** sends your new commits from your repository to the remote.
- **Pull** brings new commits from the remote into your repository.



Ada commits on her laptop and pushes. Ben pulls and gets Ada's commits. Then it's Ben's turn. The remote is the meeting point, and also a backup.

What Git Is, and Why GitHub Is Not Git

Git is the version control program you'll use in this book. It's free, it runs on your own computer, and it works entirely offline. Commits, branches, merges, and history all happen on your machine.

GitHub is a website owned by a company. It stores Git repositories on its servers, so they can serve as remotes, and it adds features around them: web pages to browse code, pull requests for reviewing changes, issue trackers, and automated checks. GitLab and Bitbucket are similar services.

The difference matters. You don't need GitHub to use Git: a folder on a shared drive can be a remote, and that's what this book's labs use. And the pull request, GitHub's best-known feature, is not part of Git at all; Chapter 11 explains it. When a tutorial says "Git", it means the program. When it says "GitHub", it means the website.



Git has many commands, but the ideas are few: snapshots, a chain of history, bookmarks called branches, merges that join them, and copies called remotes. Every command in Chapter 8 is one of these ideas in action.

Lab: Meet Git

Git needs to know who you are before it can record who made each commit. These settings live in a small file in your home folder and apply to every repository you create.

1. Check that Git is installed: `git --version`. **Expected:** a line like `git version 2.52.0`. Any version from 2.28 onward works with this book.
2. Tell Git your name and email (use your own):

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.com"
```

Expected: no output. Silence means success.

3. Make new repositories start on a branch called `main`:

```
git config --global init.defaultBranch main
```

Expected: no output.

4. Check your settings: `git config --global --list`. **Expected:**

```
user.name=Ada Lovelace
user.email=ada@example.com
init.defaultbranch=main
```

You may see extra lines if you've used Git before.

- On paper, draw the history for this story: you commit A and B on `main`; you make a branch `fix` from B and commit C on it; you commit D on `main`; then you merge `fix` into `main`, creating M. **Expected:** A ← B ← D ← M along `main`, with C branching off B and joining at M. M has two parents: D and C.



The email becomes part of every commit and is visible to anyone who sees the repository. If you'll publish code, GitHub gives you a private “noreply” address you can use here instead.

Summary, Key Terms, and Review Questions

Summary

- Version control keeps snapshots of your whole project so you can compare and return to any of them.
- A commit is a snapshot plus author, time, message, and a link to its parent. The links form the history.
- A branch is a movable bookmark pointing at a commit. HEAD marks where you are.
- A merge combines two branches; when both changed the same lines, you resolve a conflict.
- A remote is another full copy of the repository. You push to it and pull from it.
- Git is the program on your computer. GitHub is a website that hosts Git repositories and adds features like pull requests.

Key Terms

Term	Meaning
Version control	A system that records a project's states so you can return to them
Repository	The project's collected history, stored with the project
Commit	A snapshot with an author, time, message, and parent
Branch	A movable name that points at a commit
HEAD	Git's marker for "you are here"
Merge	Combining the work of two branches
Merge conflict	Both branches changed the same lines; you must choose
Remote	Another copy of the repository, usually on a server
Clone	A full copy of an existing repository
Push	Send your commits to a remote
Pull	Bring a remote's commits into your repository
Git	The free version control program that runs on your computer
GitHub	A website that hosts Git repositories and adds collaboration tools

Review Questions

1. Why is `app_final_v2.py` a poor way to keep old versions?
2. What four things does a commit record besides your files?
3. A branch "isn't a copy of your files". What is it, then, and why does that make branches cheap?
4. When does a merge need your help?
5. If GitHub disappeared tomorrow, what would happen to the repository on your laptop?

You understand this when you can draw a history with a branch and a merge, and explain every arrow and label in it.

PART II

Use



What to ignore for now: every flag of every command, advanced `awk`, and Git plumbing commands.

The Everyday Command Line

This chapter gives you the handful of tools you'll use every day: finding files, reading them, searching inside them, reshaping text, and pulling facts out of JSON. You'll then join them into pipelines that answer real questions in one line. Last, you'll meet environment variables, the settings every program inherits from the shell.

The problem. I can move around, but finding a file or reading a log still feels like guessing.

The question. How do I find what I need, in files and inside them, without opening everything?

The examples use two sample files from `labs/ch05/` in the book's lab files (see the preface): `app.log`, a web server's log, and `team.json`. From the folder that holds `labs/`, copy them into your sandbox:

```
mkdir -p ~/sandbox/logs
cp labs/ch05/app.log ~/sandbox/logs/
cp labs/ch05/team.json ~/sandbox/
```

Then `cd ~/sandbox`.

Finding Files

`find` walks down a folder tree and prints every path that matches your conditions. Its shape is `find <where> <conditions>`:

`find . -name '*.txt'` lists every `.txt` file under the current folder. The `*` is a wildcard meaning “any characters”, so `'*.txt'` matches every name ending in `.txt`. Keep the quotes; they stop the shell from expanding the `*` itself before `find` sees it. Other useful conditions:

Condition	Matches
<code>-name '*.py'</code>	Names matching a pattern
<code>-type f/-type d</code>	Only files / only directories
<code>-mtime -1</code>	Modified in the last day
<code>-size +10M</code>	Bigger than 10 megabytes

To see how much space something takes, use `du -sh` (“disk usage, summary, human units”): `du -sh logs` prints `8.0K logs`.

Viewing Files

`cat` prints a whole file, which is fine for short files and useless for long ones. For those:

- `head -n 3 file` shows the first three lines; `tail -n 3 file` the last three.
- `wc -l file` counts lines.
- `less file` opens a pager: a viewer that shows one screen at a time. Space goes forward, b goes back, /word searches, q quits.
- `tail -f file` keeps the file open and prints new lines as they’re added. It’s how you watch a running program’s log. Press Ctrl-C to stop.

```
wc -l logs/app.log
head -n 3 logs/app.log

60 logs/app.log
2026-09-25 09:14:35 INFO POST /cart 200 user=cy
2026-09-25 09:15:45 INFO POST /login 200 user=ada
2026-09-25 09:15:54 INFO GET /books 200 user=dee
```

Each line has a date, a time, a level (INFO, WARN, or ERROR), a request method and path, a status code, and a user.

Searching Inside Files with grep

grep prints the lines that contain a pattern. It's the tool you'll reach for most:

```
grep ERROR logs/app.log

2026-09-25 09:17:50 ERROR POST /cart 500 user=ada
2026-09-25 09:21:27 ERROR POST /cart 500 user=ben
2026-09-25 09:28:01 ERROR GET /books/42 500 user=ben
...
2026-09-25 09:43:30 ERROR POST /cart 500 user=cy
```

A few options cover most needs:

Option	Effect
-i	Ignore upper and lower case
-n	Show line numbers
-c	Count matching lines instead of printing them
-v	Invert: lines that don't match
-r	Search every file under a folder
-E	Use extended patterns (see below)

```
grep -c ERROR logs/app.log
grep -r milk .
```

```
8
./todo.txt:buy milk
```

The pattern is a **regular expression**: a small language for describing text. Plain words match themselves, and a few characters have special meanings:

Pattern	Matches
.	Any single character
*	Zero or more of the thing before it
^ and \$	The start and the end of a line
[0-9]	One character from a range

With -E, a vertical bar means “or”: `grep -E 'WARN|ERROR'` finds both kinds of line. And `grep -E '^2026-09-25 09:4'` finds lines that *start* with times from 09:40 to 09:49. Put patterns in single quotes so the shell leaves them alone.

Changing Text with sed

sed (stream editor) edits text as it flows past. Its most common use is find and replace, written `s/old/new/`:

```
sed 's/user=/by /' logs/app.log | head -2

2026-09-25 09:14:35 INFO POST /cart 200 by cy
2026-09-25 09:15:45 INFO POST /login 200 by ada
```

This doesn't change the file; it prints a changed copy. Add `g` at the end (`s/a/b/g`) to replace every match on a line, not just the first.

To edit a file in place, add `-i`. Here the GNU and BSD versions differ: on Linux write `sed -i 's/milk/bread/' todo.txt`; on macOS write `sed -i '' 's/milk/bread/' todo.txt`. The macOS version fails with a confusing error if you leave out the `''`.



`sed -i` overwrites the file with no undo. Run the command without `-i` first, check the output, then add `-i`.

Sorting and Counting

Three more tools turn lists into answers. `cut` picks columns: `cut -d' ' -f5` means “split on spaces and keep field 5”. `sort` puts lines in order (`-n` for numbers, `-r` to reverse). `uniq -c` collapses repeated neighbouring lines into one and counts them, which is why you always `sort` first.

Building a Pipeline

Now join them. Which pages are requested most? Build the answer one stage at a time, checking the output as you go. First, keep only the path column:

```
cut -d' ' -f5 logs/app.log | head -3

/cart
/login
/books
```

Sort the paths so duplicates sit together, count them, then sort by the count, largest first, and keep the top three:

```
cut -d' ' -f5 logs/app.log | sort | uniq -c | sort -rn | head -3

15 /cart
13 /login
12 /books
```

The same pattern answers “who hit the most errors?”:

```
grep ERROR logs/app.log | cut -d' ' -f7 | sort | uniq -c | sort -rn

4 user=ben
2 user=cy
2 user=ada
```

That’s the Unix idea from Chapter 1 at work: five small tools, one question answered, no program written.



Build pipelines one stage at a time, adding `| head` to peek at each step. When the output looks right, add the next stage.

JSON with jq

Many tools and web APIs speak **JSON**, a text format of objects in `{}` and lists in `[]`, like Python dictionaries and lists. `grep` sees JSON only as lines; `jq` understands its structure:

```
jq -r '.[].name' team.json
jq '.[] | select(.active) | .email' team.json

Ada
Ben
Cy
Dee
"ada@example.com"
"cy@example.com"
"dee@example.com"
```

`. []` means “each item in the list” and `.name` picks a field; `-r` prints plain text without quotes.

`select(.active)` keeps only items whose `active` is `true`. `jq . file.json` pretty-prints any JSON file, which alone makes it worth installing.

Environment Variables

Every program starts with a set of named settings called **environment variables**. The shell has some already: `HOME` is your home folder, `USER` your name, and `PATH` the folder list from Chapter 2. Read one with `$: echo $HOME` prints `/home/ada`.

You can create your own, but by default a variable stays inside the current shell. `export` marks it to be passed down to every program the shell starts:

```
GREETING=hello
bash -c 'echo "child sees: $GREETING"'
export GREETING
bash -c 'echo "child sees: $GREETING"'

child sees:
child sees: hello
```

Python reads them through `os.environ`. This is how programs get settings without hard-coding them: a database address, a debug switch, an API key.

Settings typed at the prompt vanish when you close the terminal. To keep them, put the `export` lines in your shell's startup file: `~/.bashrc` for `bash`, `~/.zshrc` for `zsh`. The shell runs it every time it starts.

Lab: Interrogate a Log

Copy the two sample files into `~/sandbox` as shown at the start of this chapter, then `cd ~/sandbox`.

1. Count the lines: `wc -l logs/app.log`. **Expected:** 60 `logs/app.log`.
2. Count the warnings: `grep -c WARN logs/app.log`. **Expected:** 6.
3. Show only problems: `grep -v INFO logs/app.log | wc -l`. **Expected:** 14 (six warnings and eight errors).
4. Find where Cy's requests start failing: `grep -n 'ERROR.*user=cy' logs/app.log`. **Expected:** two lines, numbered 40 and 44.
5. Which path causes the errors? `grep ERROR logs/app.log | cut -d' ' -f5 | sort | uniq -c`. **Expected:** 1 `/books/42` and 7 `/cart`.

6. Find every JSON file under your sandbox: `find ~/sandbox -name '*.json'`. **Expected:** the path to `team.json`.
7. Count the active team members: `jq '[.[] | select(.active)] | length' team.json`. **Expected:** 3.
8. Export a variable and read it from Python: `export COLOUR=blue`, then `python3 -c 'import os; print(os.environ["COLOUR"])`. **Expected:** blue.

Summary, Key Terms, and Review Questions

Summary

- `find` locates files by name, type, age, or size; `du -sh` shows how big they are.
- `head`, `tail`, `less`, and `wc` let you look at files without opening an editor. `tail -f` follows a growing log.
- `grep` finds lines matching a regular expression; `sed` rewrites text as it flows; `cut`, `sort`, and `uniq -c` turn lines into counts.
- Pipelines join these tools. Build them one stage at a time.
- `jq` reads and filters JSON.
- Environment variables pass settings to programs. `export` passes them to children; startup files make them permanent.

Key Terms

Term	Meaning
Regular expression	A pattern language for matching text
JSON	A text format of objects and lists, used by many tools and APIs
Environment variable	A named setting a program inherits when it starts

Review Questions

1. Write a `find` command that lists all Python files under your home folder.
2. What's the difference between `grep -c ERROR` and `grep ERROR | wc -l`?
3. Why must you sort before `uniq -c`?
4. Why can't your Python program see a variable you set at the prompt, and what fixes it?

You understand this when you can answer “who caused the most errors?” with a pipeline.

Programs That Keep Running

This chapter shows you running programs from the outside: how to list them, run them in the background, stop them politely or by force, and hand them to the system so they keep running after you leave. You'll also see where their logs go and how to run a job on a schedule.

The problem. My program stops the moment I close the terminal.

The question. How do I keep a program running, see what's running, and stop it cleanly?

Processes: Programs in Motion

A program is a file on disk, like a recipe in a book. A **process** is that program running, like someone cooking the recipe. The same recipe can be cooked twice at once.

The kernel gives every process a number, its **PID** (process ID), and remembers who started it, its parent. Programs you run from the shell are the shell's children. That's why closing the terminal can end your program: when the terminal goes, the shell and its children are told to hang up.

The examples use a small Python program, `labs/ch06/heartbeat.py`, which prints a line such as `14:36:53 beat 2` every two seconds until it's asked to stop.

Foreground and Background

Normally a program runs in the foreground, holding the terminal until it finishes. With `&` at the end, it runs in the background: the shell prints a job number and the PID, and gives you the prompt back:

```
python3 heartbeat.py > heartbeat.log &
jobs
```

```
[1] 25
[1]+  Running                  python3 heartbeat.py > heartbeat.log &
```

Each background program started from this shell is a job. `jobs` lists them and `fg` brings one back to the foreground. `ps` lists your processes, and `pgrep` finds one by name (on macOS, use `pgrep -fl`):

```
ps
pgrep -af heartbeat

PID TTY          TIME CMD
 21 pts/0        00:00:00 bash
 25 pts/0        00:00:00 python3
 27 pts/0        00:00:00 ps
25 python3 heartbeat.py
```

`ps aux` lists every process on the machine, and `top` shows a live view (press `q` to quit).

Jobs still belong to your terminal. To let a program survive the terminal closing, start it with `nohup` (“no hang-up”):

```
nohup python3 heartbeat.py > heartbeat.log 2>&1 &
```

`2>&1` sends errors into the same file, since no screen will show them. The log’s first line, `nohup: ignoring input`, is harmless.

Signals: Asking a Process to Stop

You talk to a running process by sending it a **signal**, a small numbered message from the kernel. The ones you’ll use:

Signal	Number	Meaning
SIGINT	2	“Interrupt.” Sent when you press Ctrl-C
SIGTERM	15	“Please finish up and stop.” The polite default
SIGKILL	9	“Stop now.” Cannot be caught or ignored

Signals are sent with `kill`, a gloomy name for something usually polite. `kill 25` sends SIGTERM to PID 25; `kill %1` sends it to job 1. A well-written program catches

SIGTERM, finishes what it was doing, saves its work, and exits. That's a graceful shutdown. `heartbeat.py` does it by registering a Python function to run when SIGTERM arrives:

```
def stop(signum, frame):
    global running
    print(f"got signal {signum}, finishing up", flush=True)
    running = False

signal.signal(signal.SIGTERM, stop)
```

The main loop checks `running`, so it stops after the current beat. Send SIGTERM with `kill %1`, and a couple of seconds later the shell reports `[1]+ Done`, and the log ends with a good-bye:

```
tail -n 3 heartbeat.log

14:36:55 beat 3
got signal 15, finishing up
saved my work, bye
```

`kill -9` sends SIGKILL instead. The kernel ends the process on the spot, the shell reports `Killed`, and the program gets no chance to save anything.



Use `kill -9` only when a polite `kill` hasn't worked after a few seconds. A program killed mid-write can leave a half-written, corrupted file.

Services: Programs the System Looks After

With `nohup`, nobody restarts a crashed program, and a reboot ends it. For programs that should always run, the system takes charge.

A **service** (or daemon) is a background program with no terminal, supervised by the **init system**: the first process the kernel starts at boot, PID 1. It starts services, restarts them if they crash, and collects their output. On most Linux systems it's `systemd`; on macOS it's `launchd`, controlled with `launchctl`.

With `systemd`, you describe a service in a small text file called a unit file. For a personal service, it goes in `~/.config/systemd/user/heartbeat.service`:

```
[Unit]
Description=Heartbeat demo

[Service]
ExecStart=/usr/bin/python3 /home/ada/sandbox/heartbeat.py
Restart=on-failure

[Install]
WantedBy=default.target
```

`ExecStart` is the command, `Restart=on-failure` restarts it after a crash, and `WantedBy` starts it automatically. You control it with `systemctl`:

```
systemctl --user daemon-reload
systemctl --user enable --now heartbeat
systemctl --user status heartbeat
```

- `heartbeat.service - Heartbeat demo`
Active: active (running) since Fri 2026-09-25 10:02:11 UTC
Main PID: 2143 (python3)
...

`stop` sends `SIGTERM`, `restart` stops and starts it, and `disable` stops it starting automatically. System-wide services use `sudo` instead of `--user`.



`systemd` runs only on Linux, and not inside every WSL setup or container. On macOS, use `nohup` for this chapter's lab, and read this section for the idea.

Logs: Where the Output Goes

A service has no screen, so its output goes into a **log**: a record of timestamped messages. `systemd` keeps every service's output in its journal, and `journalctl -f` follows it like `tail -f`:

```
journalctl --user -u heartbeat -f
```

Many programs write their own log files, traditionally under `/var/log`; read them with the tools from Chapter 5. On macOS, use the Console app.

Scheduled Jobs with cron

Some work should run only at set times, like a nightly backup. **cron** is the classic scheduler on Linux and macOS. Each user has a crontab, a table of jobs, edited with `crontab -e` and listed with `crontab -l`. Each line has five time fields, then the command:

```

minute (0-59)
hour (0-23)
day of month (1-31)
month (1-12)
day of week (0-6, Sunday is 0)
0 7 * * 1-5 /home/ada/bin/morning.sh

```

A `*` means “every”. This line runs `morning.sh` at 07:00, Monday to Friday.

cron runs jobs with a bare environment, without your `PATH` or `~/.bashrc`. Use full paths, and send output to a file (`>> /home/ada/morning.log 2>&1`), or you’ll never see the errors. Chapter 9 schedules a real script.

Lab: A Program That Outlives the Terminal

Copy `labs/ch06/heartbeat.py` into `~/sandbox` and `cd` there.

1. Run it in the foreground: `python3 heartbeat.py`. After a few beats, press `Ctrl-C`. **Expected:** beats every two seconds, then a traceback ending in `KeyboardInterrupt`. `Ctrl-C` sent `SIGINT`, which this program doesn’t handle gracefully.
2. Start it in the background: `nohup python3 heartbeat.py > heartbeat.log 2>&1 &`. **Expected:** a job number and PID, such as `[1] 25`, and your prompt back.
3. Find it: `pgrep -af heartbeat` (macOS: `pgrep -fl heartbeat`). **Expected:** the same PID and the command.
4. Watch it with `tail -f heartbeat.log`, then press `Ctrl-C`. **Expected:** new beats appear; stopping `tail` doesn’t stop the program.
5. Close the terminal window, open a new one, and run `pgrep -af heartbeat` again. **Expected:** it’s still running, under the same PID.
6. Stop it gracefully: `kill <PID>`, wait a few seconds, then `tail -n 2 ~/sandbox/heartbeat.log`. **Expected:** got signal 15, finishing up and saved my work, bye.

Summary, Key Terms, and Review Questions

Summary

- A process is a running program with a PID and a parent. `ps`, `pgrep`, and `top` show them.
- `&` runs a command in the background; `jobs` and `fg` manage it. `nohup` lets it survive the terminal closing.
- Signals ask processes to act. `SIGTERM` asks politely; `SIGKILL` forces. Prefer graceful shut-down.
- Services are supervised by the init system (`systemd` on Linux, `launchd` on macOS), which starts, restarts, and logs them.
- Logs hold what background programs say. `journalctl` reads `systemd`'s; `tail -f` follows a file.
- `cron` runs commands on a schedule, with a bare environment.

Key Terms

Term	Meaning
Process	A running instance of a program
PID	The number the kernel gives each process
Signal	A small numbered message sent to a process
Service	A background program supervised by the system
Init system	The first process (PID 1), which starts and supervises services
Log	A record of timestamped messages written by a program
cron	The classic scheduler for running commands at set times

Review Questions

1. What's the difference between a program and a process?
2. Why does a program started normally stop when you close its terminal? Name two ways to prevent that.
3. Why should you try `kill` before `kill -9`?
4. What does a service manager give you that `nohup` doesn't?
5. A `cron` job works when you run it by hand but fails from `cron`. What's the likely cause?

You understand this when you can start a program in the background, find its PID, read its output, and stop it gracefully.

Installing Software and Working Remotely

This chapter covers two everyday jobs: getting the software you need, safely and in the right version, and working on a machine that isn't in front of you, with SSH keys, file copies, and sessions that survive a dropped connection.

The problem. I need a tool on a machine I can only reach over the internet.

The question. How do I install tools safely, and work on a machine that isn't in front of me?

Package Managers

On a phone you use the app store, not random websites. The terminal's app store is a **package manager**. It downloads software from a trusted source, installs it, and can update or remove it later. Each piece of software is a package, and most need other packages, their dependencies, which the package manager installs for you.

System	Package manager	Install	Update everything
Ubuntu, Debian, WSL	apt	<code>sudo apt install jq</code>	<code>sudo apt update && sudo apt upgrade</code>
Fedora, Red Hat	dnf	<code>sudo dnf install jq</code>	<code>sudo dnf upgrade</code>
macOS	Homebrew	<code>brew install jq</code>	<code>brew update && brew upgrade</code>

On Debian and Ubuntu, `apt update` only refreshes the list of what's available; `apt upgrade` installs newer versions. Run `update` before installing. Homebrew, installed from `brew.sh`, keeps everything in your own folders, so it needs no `sudo`.



Some guides say to paste `curl ... | sudo bash`, which runs a downloaded script as root, unread. Prefer your package manager, or download the script and read it first.

Virtual Environments for Python

Project A needs version 1 of a library and project B needs version 2, but your system has one Python. A **virtual environment** solves this: a folder with its own Python and its own packages, one per project. Create one in the project folder, then activate it:

```
python3 -m venv .venv
source .venv/bin/activate
which python

/home/ada/sandbox/.venv/bin/python
```

Activating puts `.venv/bin` at the front of your `PATH`, so `python` and `pip` mean the project's copies, and your prompt shows `(.venv)`. Now `pip install` requests installs into this project only. `deactivate` goes back. A version manager, such as `pyenv` or `uv` for Python, goes further and installs whole language versions side by side.



Keep `.venv` out of Git (Chapter 8 shows how). Record what the project needs with `pip freeze > requirements.txt` instead, so anyone can rebuild the environment.

SSH: A Terminal on Another Machine

SSH (Secure Shell) lets you open a shell on another computer over the network. Everything you type and see travels encrypted.

```
ssh ada@203.0.113.10
```

That means “log in as `ada` on the machine at that address”. The first time, SSH shows the server's fingerprint and asks whether to trust it; it remembers your answer in `~/.ssh/known_hosts` and warns loudly if the fingerprint ever changes.

You could log in with a password, but passwords can be guessed. The better way is a **key pair**: two linked files. Think of a padlock and its key. The **public key** is the padlock: you can hand out copies freely and fit one to every server you use. The **private key** is the only key that opens those padlocks, and it never leaves your laptop.

```
ssh-keygen -t ed25519 -C "ada@laptop" -f ~/.ssh/id_ed25519
```

It asks for a passphrase twice. Choose one: it protects the private key if your laptop is stolen. Then:

```
Your identification has been saved in /home/ada/.ssh/id_ed25519
Your public key has been saved in /home/ada/.ssh/id_ed25519.pub
...
```

The .pub file is the padlock:

```
cat ~/.ssh/id_ed25519.pub

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDIpXo4KbS5P4qftLjgXDY393xIkLDJdG/ggTPq9Dqyk
↪ ada@laptop
```

`ssh-copy-id ada@203.0.113.10` installs it on a server; GitHub and cloud providers have a page where you paste it. From then on, ssh logs you in with the key.



Never share or copy `~/.ssh/id_ed25519` (the file without `.pub`). Anyone who has it can log in as you everywhere you've installed the padlock. `ssh-keygen` makes it 600 for you; keep it that way.

To save typing, name your servers in `~/.ssh/config`:

```
Host web
  HostName 203.0.113.10
  User ada
```

Now `ssh web` is enough.

Copying Files Between Machines

`scp` copies files over SSH, like `cp` with a remote path written `host:path`: `scp report.csv web:/home/ada/`. For folders you copy again and again, use `rsync`, which sends only what changed. It works between local folders too:

```
rsync -av notes/ backup/notes/
```

It lists each file it copies. Add a file to notes and run the same command again, and only the new file is sent. The trailing `/` on `notes/` means “the contents of notes”.

Keeping Work Alive with tmux

If your connection drops, the shell on the server ends, and so does anything running in its foreground. `tmux` runs sessions on the server that keep going while you're away. Start one with `tmux new -s work`, run your long job, then press `Ctrl-B` followed by `D` to detach. Later, from a new login, `tmux attach -t work` puts you back exactly where you were. Install it with `apt install tmux` or `brew install tmux`.

Lab: Your Toolbox

Work in `~/sandbox`. None of these steps needs a server.

1. Install `jq` and `shellcheck` with your package manager if you haven't: `sudo apt install jq shellcheck` or `brew install jq shellcheck`. **Expected:** `jq --version` prints a version such as `jq-1.8.2`.
2. Create and activate a virtual environment: `python3 -m venv .venv` then `source .venv/bin/activate`. **Expected:** your prompt starts with `(.venv)`, and which `python` points inside `~/sandbox/.venv`.
3. Run `pip list`, then deactivate. **Expected:** only `pip` is listed, then `(.venv)` disappears.
4. Make a practice key pair: `ssh-keygen -t ed25519 -C "practice" -f ~/sandbox/practice_key`. **Expected:** two files, `practice_key` (mode `-rw-----`) and `practice_key.pub`. Check with `ls -l practice_key*`.
5. Read the public half: `cat practice_key.pub`. **Expected:** one line starting with `ssh-ed25519`. Delete both files when you're done.
6. Sync a folder twice: `mkdir -p notes`, `echo a > notes/monday.txt`, `rsync -av notes/ backup/notes/`; then `echo c > notes/wednesday.txt` and run the same `rsync` again. **Expected:** the second run copies only `wednesday.txt` (it may also list `.`, the folder itself).

Summary, Key Terms, and Review Questions

Summary

- Package managers install software and its dependencies from trusted sources: `apt`, `dnf`, Homebrew.
- A virtual environment gives each Python project its own packages; version managers install whole language versions.

- SSH opens an encrypted shell on another machine. Key pairs beat passwords: share the public key, guard the private key.
- `scp` copies files over SSH; `rsync` copies only what changed.
- `tmux` keeps sessions alive when your connection drops.

Key Terms

Term	Meaning
Package manager	A tool that installs, updates, and removes software
Virtual environment	A per-project folder with its own Python and packages
SSH	Secure Shell: an encrypted way to use another machine's shell
Key pair	A linked public and private key used to prove who you are
Public key	The half you share, installed on servers like a padlock
Private key	The secret half that stays on your machine

Review Questions

1. What's the difference between `apt update` and `apt upgrade`?
2. Why does each Python project deserve its own virtual environment?
3. In the padlock picture, which file do you give to a server, and which do you never share?
4. When would you choose `rsync` over `scp`?
5. You start a two-hour job over SSH and your Wi-Fi drops. How could `tmux` have saved it?

You understand this when you can set up a project's Python environment, and explain how an SSH key lets you in without a password.

Git Every Day

This chapter turns the ideas from Chapter 4 into commands. You'll start a repository, learn where your changes live before and after a commit, read history and differences, keep unwanted files out, work on branches, share with a remote, and undo the most common mistakes. These are the Git commands you'll use every working day.

The problem. I want to save my work properly, not as `final_v2_really_final.py`.

The question. What are the daily Git commands, and what does each one do to my project?

The examples build a tiny project, `recipes`, that scales ingredient amounts. You need the name and email you set in the Chapter 4 lab.

Starting a Repository

Any folder becomes a Git repository with `git init`:

```
mkdir -p ~/projects/recipes
cd ~/projects/recipes
git init
```

```
Initialized empty Git repository in /home/ada/projects/recipes/.git/
```

Git created one hidden folder, `.git`. That's the repository: every commit and branch lives inside it. Your files stay where they are. Delete `.git` and the folder is an ordinary folder again, with all its history gone.

`git status` is the command you'll run most. It tells you where you are and what has changed:

```
git status

On branch main

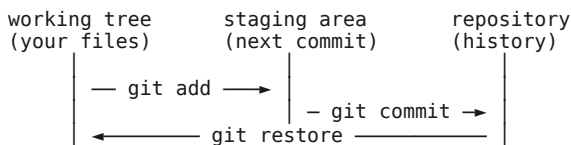
No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Three Places Your Work Lives

Git keeps your work in three places, and every command moves changes between them. Think of posting a parcel. Your desk is where you make things. The box is where you put exactly what you want to send. The post office stamps the box and keeps a record forever.

- The **working tree** is your desk: the normal files you edit.
- The **staging area** is the box: the changes you've chosen for the next commit. (Git also calls it the index.)
- The repository is the post office: the commits, stored safely in `.git`.



Why the box? Because you often change several things at once, and they belong in separate commits. Staging lets you choose.

The Everyday Loop: Add and Commit

Create `scale.py` in your editor:

```
def scale(amount, factor):
    return amount * factor

print(scale(200, 2))
```

Git notices the new file but doesn't track it yet: `git status` now lists `scale.py` under “Untracked files”. `git add` puts it in the box; `git commit` records the snapshot. The `-m` gives the message:

```
git add scale.py
git commit -m "Add function to scale a recipe"

[main (root-commit) 4978b71] Add function to scale a recipe
1 file changed, 4 insertions(+)
create mode 100644 scale.py
```

4978b71 is the start of the new commit's ID. `git log` shows the history: each commit's full ID, author, date, and message.

That's the loop you'll repeat all day: edit, `git status`, `git add`, `git commit`.



`git add .` stages every change in the current folder. It's handy, but run `git status` first so you don't commit something you didn't mean to.

Seeing What Changed

Now change `scale.py` so it rounds results, and add a second example:

```
def scale(amount, factor):
    return round(amount * factor, 1)

print(scale(200, 2))
print(scale(150, 0.333))
```

`git diff` shows changes in the working tree that aren't staged yet. Lines starting with `-` were removed; lines starting with `+` were added:

```
git diff

diff --git a/scale.py b/scale.py
...
@@ -1,4 +1,5 @@
     def scale(amount, factor):
-        return amount * factor
+        return round(amount * factor, 1)

     print(scale(200, 2))
+print(scale(150, 0.333))
```

Once you `git add` the file, plain `git diff` shows nothing, because nothing is left unstaged. `git diff --staged` shows what's in the box instead. Always look before you commit:

```
git add scale.py
git commit -m "Round scaled amounts to one decimal place"
git log --oneline

[main 450c3a8] Round scaled amounts to one decimal place
 1 file changed, 2 insertions(+), 1 deletion(-)
450c3a8 Round scaled amounts to one decimal place
4978b71 Add function to scale a recipe
```

`git log --oneline` gives one line per commit, newest first. `git show 450c3a8` shows a single commit and its changes.

Keeping Files Out with .gitignore

Some files must never be committed: secrets such as `.env`, and files your tools generate, such as Python's `__pycache__` folder or a `.venv`. Git lists them as untracked, which is noise at best and a leak at worst:

```
git status --short

?? .env
?? __pycache__/
```

List the patterns to ignore in a file called **.gitignore** at the top of the project, one per line:

```
.env
__pycache__/
.venv/
```

Now `git status --short` shows only `?? .gitignore`. Commit the `.gitignore` itself, so everyone who clones the project ignores the same things.



`.gitignore` only stops files from being added. If a secret was already committed, ignoring it now doesn't remove it from history. Chapter 12 explains what to do.

Writing Good Commit Messages

A commit message is a note to the person reading the history later, usually you. Write the first line as a short command that completes the sentence “If applied, this commit will...”. Keep it under about 50 characters. If more explanation helps, leave a blank line and write why you made the change.

Weak	Better
fix	Fix crash when amount is zero
changes	Round scaled amounts to one decimal place
stuff for Sam	Add ounces-to-grams conversion

One commit should hold one logical change. If your message needs “and”, consider two commits.

Branches and Merging

To try something without touching `main`, create a branch and switch to it in one step with `git switch -c`. `git branch` lists branches; the `*` marks where HEAD is:

```
git switch -c metric
git branch
```

```
Switched to a new branch 'metric'
main
* metric
```

Add a `to_grams` function to `scale.py` and commit it on the branch. (`git commit -am` stages every changed tracked file and commits in one step.) Then switch back: `main` doesn't have the new function yet, because the commit lives only on `metric`:

```
git commit -am "Add ounces-to-grams conversion"
git switch main
git merge metric
```

```
[metric 015db0b] Add ounces-to-grams conversion
 1 file changed, 4 insertions(+)
Switched to branch 'main'
Updating f88f44f..015db0b
Fast-forward
 scale.py | 4 ++++
 1 file changed, 4 insertions(+)
```

This was a **fast-forward** merge. `main` hadn't moved since the branch split, so Git simply slid the `main` bookmark forward to the branch's commit. No new commit was needed. Delete the finished branch with `git branch -d metric`.

When both branches have new commits, Git makes a merge commit with two parents. Git opens your editor with a ready-made message; save and close it to finish. `git log --graph` draws the result:

```
* 0ea3aaa Merge branch 'readme'
|\
| * 97e1215 Add a README
* | 8ea3115 Print a conversion example
|/
* 015db0b Add ounces-to-grams conversion
```



If Git opens an editor you don't know how to leave, you're probably in Vim: type `:wq` and press Enter. To use a friendlier editor, run `git config --global core.editor "nano"` (or `"code --wait"` for VS Code).

Push and Pull

A remote can be any Git repository you can reach, including a folder on your own disk. A **bare repository** has history but no working tree: nobody edits files in it, it only receives and hands out commits. That's what hosting services like GitHub keep on their servers, and you can make one yourself:

```
git init --bare ~/remotes/recipes.git
git remote add origin ~/remotes/recipes.git
git push -u origin main
```

```
Initialized empty Git repository in /home/ada/remotes/recipes.git/
To /home/ada/remotes/recipes.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

`git remote add` gives the remote a short name; `origin` is the usual name for the main one. `git push -u origin main` sends your main branch there, and `-u` remembers the pairing, so later you can type just `git push` and `git pull`.

Anyone (including you, on another machine) can now `git clone ~/remotes/recipes`. `git`, `commit`, and `git push`. To bring their commits into your copy, `pull`:

```
git pull

From /home/ada/remotes/recipes
 0ea3aaa..1d39f76  main      -> origin/main
Updating 0ea3aaa..1d39f76
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

Chapter 11 covers what happens when two people push at once.

Undoing Mistakes

Git can undo almost anything, as long as you pick the right tool for where the mistake lives:

Situation	Command
Throw away edits to a file (not yet staged)	<code>git restore scale.py</code>
Take a file out of the box, keeping the edits	<code>git restore --staged notes.txt</code>
Fix the last commit's message, or add a forgotten file	<code>git commit --amend</code>
Undo a commit that's already pushed	<code>git revert <id></code>
Undo the last commit, keeping its changes staged	<code>git reset --soft HEAD~1</code>
Throw away the last commit and its changes	<code>git reset --hard HEAD~1</code>

HEAD~1 means “one commit before where I am”. A typo'd message is fixed like this:

```
git commit -am "Add serving size"
git commit --amend -m "Add serving size"
```

`git revert` is the safe undo for shared history. Instead of deleting a commit, it adds a new commit that does the opposite, so nobody who already pulled is confused:

```
git revert --no-edit HEAD
git log --oneline -3

[main 2f795a7] Revert "Add serving size"
...
2f795a7 Revert "Add serving size"
ebeb8c Add serving size
1d39f76 Describe the project
```

`git reset` moves the branch bookmark backwards, as if later commits never happened. Use it only on commits you haven't pushed.



`git restore <file>` and `git reset --hard` discard uncommitted work for good. Git never saw those edits, so it can't give them back. Commit often.

And if you reset too far? Suppose you ran `git reset --hard HEAD~1` twice, throwing away both the revert and “Add serving size”. Git keeps a diary of everywhere HEAD has been, the **reflog**, and even “lost” commits appear there for weeks:

```
git reflog -4
```

```
1d39f76 HEAD@{0}: reset: moving to HEAD~1
ebeb8c HEAD@{1}: reset: moving to HEAD~1
2f795a7 HEAD@{2}: revert: Revert "Add serving size"
ebeb8c HEAD@{3}: commit (amend): Add serving size
```

`git reset --hard ebeb8c` would bring that commit back. Chapter 10 practises exactly this rescue.

Lab: Save Your Work Properly

Make a new folder and go into it: `mkdir -p ~/projects/lab8`, then `cd ~/projects/lab8`.

1. `git init`, then `git status`. **Expected:** On branch main and No commits yet.
2. Create `hello.py` containing `print("hello")`. Run `git status --short`. **Expected:** ?? `hello.py`.
3. `git add hello.py`, then `git status --short`. **Expected:** A `hello.py`.
4. `git commit -m "Add greeting script"`. **Expected:** a line starting [main (root-commit) and 1 file changed.
5. Change the text to "hello, world" and run `git diff`. **Expected:** one - line and one + line.
6. `git commit -am "Greet the world"`, then `git log --oneline`. **Expected:** two commits, newest first.
7. Create `.env` and a `.gitignore` containing `.env`. Run `git status --short`. **Expected:** only ?? `.gitignore`. Commit it with `git add .gitignore` and `git commit -m "Ignore .env"`.
8. `git switch -c shout`, change the text to "HELLO, WORLD", commit, `git switch main`, `git merge shout`. **Expected:** Fast-forward, and `python3 hello.py` prints HELLO, WORLD.
9. Make a bare remote with `git init --bare ~/remotes/lab8.git`, then `git remote add origin ~/remotes/lab8.git` and `git push -u origin main`. **Expected:** * [new branch] main -> main.
10. Edit `hello.py` badly, then `git restore hello.py`. **Expected:** `git status` says nothing to commit, working tree clean.

Summary, Key Terms, and Review Questions

Summary

- `git init` creates the repository (`.git`). `git status` is your compass.
- Work moves from the working tree, to the staging area (`git add`), to the repository (`git commit`).
- `git diff`, `git diff --staged`, `git log`, and `git show` let you see changes and history.
- `.gitignore` keeps secrets and generated files out. Good messages say what the commit does, in a short command.
- `git switch -c` creates a branch; `git merge` joins it back, by fast-forward or with a merge commit.
- `git push` and `git pull` sync with a remote such as `origin`.
- Undo with `restore`, `--amend`, `revert` (safe for shared history), or `reset` (local only). The reflog remembers where you've been.

Key Terms

Term	Meaning
Working tree	The project files you see and edit
Staging area	The changes chosen for the next commit (also called the index)
<code>.gitignore</code>	A file listing patterns Git should not track
Fast-forward	A merge that just moves the branch pointer forward
Bare repository	A repository with history but no working tree, used as a remote
Reflog	Git's record of everywhere HEAD has pointed

Review Questions

1. Name the three places your work lives, and the commands that move it.
2. What's the difference between `git diff` and `git diff --staged`?
3. Why ignore `.env`, and why isn't that enough once it's committed?
4. When does a merge fast-forward, and when does it create a merge commit?
5. You pushed a bad commit an hour ago. Should you use `reset` or `revert`? Why?

You understand this when you can make a change, review it with `git diff`, commit it with a clear message, and undo it the safe way.

PART III

Apply



What to ignore for now: Git hooks, submodules, and complex branching strategies.

Lab: Automate a Daily Chore

In this lab you turn five commands you'd type every morning into a shell script you can trust to run on its own. Along the way you meet everything a beginner needs for scripting: the shebang, a safety line that stops a script at the first error, variables and quoting, conditions, loops, arguments, exit codes, a checker that finds bugs for you, and a schedule.

The problem. I type the same five commands every morning.

The question. How do I turn commands I type into a script I can trust to run by itself?

The finished script is in `labs/ch09/morning.sh`. Type it yourself as you go, and use the copy only to check your work.

Step 1: Do the Chore by Hand

Ada keeps plain-text notes in a `notes` folder, one file per day. Every morning she creates today's note, backs up the folder to a `backups` folder, deletes backups older than a week, and checks how much she has written. Set up the same folders inside your sandbox, so nothing touches your real files:

```
mkdir -p ~/sandbox/notes ~/sandbox/backups
printf '# Ideas\nLearn Git branches.\nTry tmux.\n' > ~/sandbox/notes/ideas.md
```

Now the five commands. `$(date +%F)` is replaced by today’s date, such as `2026-09-26`; this is called command substitution:

```
cd ~/sandbox/notes
echo "# Notes for $(date +%F)" > "$(date +%F).md"
tar -czf ~/sandbox/backups/notes-$(date +%F).tar.gz .
find ~/sandbox/backups -name 'notes-*.tar.gz' -mtime +7 -delete
wc -w *.md

4 2026-09-26.md
7 ideas.md
11 total
```

`tar -czf` packs the folder into one compressed file, and `find ... -mtime +7 -delete` removes backups more than seven days old.

Expected: a new note named after today’s date, and a backup in `~/sandbox/backups`. Check with `ls ~/sandbox/backups`.

Step 2: Put the Commands in a File

A **script** is just commands saved in a file. Create `~/sandbox/morning.sh` in your editor with the same five lines, plus one line at the top:

```
#!/usr/bin/env bash
cd ~/sandbox/notes
echo "# Notes for $(date +%F)" > "$(date +%F).md"
tar -czf ~/sandbox/backups/notes-$(date +%F).tar.gz .
find ~/sandbox/backups -name 'notes-*.tar.gz' -mtime +7 -delete
wc -w *.md
```

The first line is the **shebang**. `#!` tells the kernel which program should run this file; `/usr/bin/env bash` means “whichever bash is first on my PATH”. Now make it executable and run it, as in Chapter 3:

```
cd ~/sandbox
chmod +x morning.sh
./morning.sh
```

Expected: the same word counts as Step 1. The `./` matters: your sandbox isn’t on your PATH, so you must say where the script is.

Step 3: Watch It Fail Badly

Scripts run without you watching, so ask: what if something goes wrong? Rename the notes folder, then run the script from an empty folder so the damage stays small:

```
mv ~/sandbox/notes ~/sandbox/notes-old
mkdir -p ~/sandbox/empty
cd ~/sandbox/empty
~/sandbox/morning.sh
echo $?

/home/ada/sandbox/morning.sh: line 2: cd: /home/ada/sandbox/notes: No such file or
↳ directory
4 2026-09-26.md
0
```

The `cd` failed, and the script carried on regardless. It created a stray note in the empty folder, backed up that folder instead of your notes, and then reported success. Run from your home folder, it would have backed up your entire home folder. `$?` holds the **exit code** of the last command: `0` means success, anything else means failure. Every command sets one, and so does every script.

Clean up the damage: `rm ~/sandbox/empty/*.md ~/sandbox/backups/notes-$(date +%F).tar.gz`. Leave the notes folder renamed for Step 4.

Step 4: Add the Safety Line

Add this as the second line of the script, right after the shebang:

```
set -euo pipefail
```

This is often called **strict mode**. `-e` stops the script at the first failing command. `-u` treats a misspelled or unset variable as an error instead of silently using an empty string. `-o pipefail` makes a pipeline fail if any stage fails, not just the last one. Repeat Step 3's experiment:

```
/home/ada/sandbox/morning.sh: line 3: cd: /home/ada/sandbox/notes: No such file or
↳ directory
1
```

Expected: the script stops at `cd`, touches nothing, and returns 1. Put the notes folder back before continuing: `cd ~/sandbox, then mv notes-old notes`.



Start every bash script with `set -euo pipefail`. It turns silent disasters into loud, early stops.

Step 5: Use Variables, and Quote Them

Repeating `~/sandbox/notes` and `$(date +%F)` is fragile. Store them in a **variable** instead: a name that holds a value. Write it with no spaces around `=`, and read it back with `$`:

```
notes_dir="$HOME/sandbox/notes"
today="$(date +%F)"
note="$notes_dir/$today.md"
```

Now quoting. Always put double quotes around a variable when you use it. Here's why. Suppose the folder were called `my notes`:

```
notes_dir="$HOME/sandbox/my notes"
ls $notes_dir

ls: cannot access '/home/ada/sandbox/my': No such file or directory
notes:
2026-09-26.md  ideas.md
```

Without quotes, the shell split the value at the space into two arguments, `/home/ada/sandbox/my` and `notes`. The first didn't exist. The second happened to match a different folder (you ran this from `~/sandbox`), which `ls` listed as if nothing were wrong. With `rm` instead of `ls`, that's how scripts delete the wrong files. Written as `ls "$notes_dir"`, the value stays one argument.

Double quotes still expand variables; single quotes don't. `echo "$HOME"` prints your home folder; `echo '$HOME'` prints `$HOME`. And `-u` catches typos. Here a tiny script, run with `bash -c`, misspells `notes_dir`:

```
bash -c 'set -u; notes_dir=~ /sandbox/notes; ls "$note_dir"'

bash: line 1: note_dir: unbound variable
```

Step 6: Make Decisions with if

Running the script twice would overwrite today's note. A script that's safe to run again and again, leaving the same result, is **idempotent**, and scheduled scripts should always be. An `if` statement makes it so:

```
if [[ -f "$note" ]]; then
    echo "Today's note already exists"
else
    echo "# Notes for $today" > "$note"
    echo "Created $note"
fi
```

`[[ ... ]]` is a test. Common ones:

Test	True when
<code>-f "\$x"</code>	<code>\$x</code> is an existing file
<code>-d "\$x"</code>	<code>\$x</code> is an existing folder
<code>-z "\$x"</code>	<code>\$x</code> is empty
<code>"\$a" == "\$b"</code>	The two strings are equal
<code>! ...</code>	The test after it is false

Spaces inside the brackets are required: `[[ -f "$note" ]]` is an error.

Step 7: Repeat with a Loop

Replace `wc -w *.md` with a `for` loop that reports each note on its own line. The loop runs its body once per matching file, with `file` holding each name in turn:

```
for file in "$notes_dir"/*.md; do
    echo "$(wc -w < "$file") words in $(basename "$file")"
done
```

`basename` strips the folder from a path, leaving just the file name.

Step 8: Accept an Argument and Fail Clearly

A script can take **arguments** just like a command. Inside, the first is `$1`, the second `$2`. `${1:-$HOME/sandbox/notes}` means “the first argument, or `~/sandbox/notes` if none was given”. If the folder is missing, the script should say so on `stderr` (`>&2`) and exit with a non-zero code, so whatever runs it knows it failed. Here is the finished script:

```
#!/usr/bin/env bash
# morning.sh: start the day. Make today's note, back up all notes,
# and delete backups older than a week.
set -euo pipefail

notes_dir="${1:-$HOME/sandbox/notes}"
backup_dir="$HOME/sandbox/backups"
today="$(date +%F)"
note="$notes_dir/$today.md"

if [[ ! -d "$notes_dir" ]]; then
    echo "morning: no notes folder at $notes_dir" >&2
    exit 1
fi

if [[ -f "$note" ]]; then
    echo "Today's note already exists"
else
    echo "# Notes for $today" > "$note"
    echo "Created $note"
fi

mkdir -p "$backup_dir"
archive="$backup_dir/notes-$today.tar.gz"
tar -czf "$archive" -C "$notes_dir" .
echo "Backed up to $archive"

find "$backup_dir" -name 'notes-*.tar.gz' -mtime +7 -print -delete

for file in "$notes_dir"/*.md; do
    echo "$(wc -w < "$file") words in $(basename "$file")"
done
```

`tar -C "$notes_dir"` means “change into that folder first”, so the script no longer needs `cd` at all. `-print` makes `find` name each backup it deletes. Run it twice:

```
Created /home/ada/sandbox/notes/2026-09-26.md
Backed up to /home/ada/sandbox/backups/notes-2026-09-26.tar.gz
4 words in 2026-09-26.md
7 words in ideas.md
Today's note already exists
Backed up to /home/ada/sandbox/backups/notes-2026-09-26.tar.gz
4 words in 2026-09-26.md
7 words in ideas.md
```

The first run created the note; the second changed nothing it shouldn't. Once you have back-

ups more than a week old, their paths appear after the “Backed up” line as they’re deleted. (On macOS, wc pads the numbers with spaces.) Now give it a folder that doesn’t exist:

```
./morning.sh /nope
echo $?

morning: no notes folder at /nope
1
```

Exit codes let you chain commands: `a && b` runs `b` only if `a` succeeded, and `a || b` runs `b` only if `a` failed. So `./morning.sh || echo "morning failed"` prints a warning when something goes wrong.

Step 9: Let shellcheck Review It

shellcheck reads a script and points out common bugs, especially missing quotes. Run it on the first version from Step 2:

```
shellcheck morning.sh

In morning.sh line 2:
cd ~/sandbox/notes
^-----^ SC2164 (warning): Use 'cd ... || exit' or 'cd ... || return' in
↳ case cd fails.

...
In morning.sh line 4:
tar -czf ~/sandbox/backups/notes-$(date +%F).tar.gz .
                                     ^-----^ SC2046 (warning): Quote this to
↳ prevent word splitting.
```

It found exactly the two problems you fixed by hand: the unchecked `cd` and the unquoted substitution. On the finished script it prints nothing, which, as always, means success. Run shellcheck on every script you write. To see each command as it runs, use `bash -x morning.sh`.

Step 10: Schedule It

Finally, let the computer remember. Open your crontab with `crontab -e` and add one line (Chapter 6 explains the five time fields):

```
0 7 * * 1-5 /home/ada/sandbox/morning.sh >> /home/ada/sandbox/morning.log 2>&1
```

At 07:00 every weekday, cron runs the script and appends everything it prints, including errors, to `morning.log`. Use full paths: cron doesn't read your shell's startup files. Check the log the next morning.



On macOS, cron works but may need permission to reach your files (System Settings, Privacy & Security, Full Disk Access). macOS's own scheduler is `launchd`. On WSL, cron runs only while WSL itself is running. Remove the line with `crontab -e` when you're done with this lab.

Summary, Key Terms, and Review Questions

Summary

- A script is commands in a file. The shebang picks the interpreter; `chmod +x` makes it runnable.
- `set -euo pipefail` stops a script at the first error, unset variable, or failed pipeline stage.
- Store values in variables, and always double-quote them when you use them.
- `if [[ ... ]]` makes decisions; `for` repeats work; `$1` reads arguments.
- Every command returns an exit code: 0 for success. `exit 1` and `>&2` report failure properly.
- Scheduled scripts should be idempotent. `shellcheck` catches bugs before they bite, and cron runs the script on time.

Key Terms

Term	Meaning
Script	A file of commands the shell runs in order
Shebang	The <code>#!</code> first line naming the program that runs a script
Exit code	A number every command returns: 0 success, non-zero failure
Strict mode	<code>set -euo pipefail</code> : stop on errors, unset variables, and failed pipes
Variable	A name that holds a value, read with <code>\$name</code>
Idempotent	Safe to run many times with the same result
Argument	A value passed to a script, read as <code>\$1</code> , <code>\$2</code> , ...

Review Questions

1. What does the shebang line do, and why `/usr/bin/env bash` rather than a fixed path?
2. In Step 3, the script “succeeded” while doing damage. Which part of strict mode prevents that?
3. Why does `ls $dir` misbehave when `dir` contains a space, and what’s the fix?
4. What makes a script idempotent, and why does that matter for scheduled jobs?
5. What does `./morning.sh || echo "failed"` print when the script succeeds? When it fails?

You understand this when you can take any sequence of commands you type often and turn it into a strict, quoted, idempotent script that shellcheck approves of.

Lab: Try an Idea Without Breaking Anything

In this lab you use a branch as a safe place to experiment. You'll hit a real merge conflict and resolve it, see what rebasing does to history, and rescue a commit you thought you'd deleted. After this, branches stop being scary and become the first thing you reach for.

The problem. I want to try an idea without risking the version that works.

The question. How do I experiment on a copy of my code and bring back only what works?

Step 1: A Project That Works

Make a new repository with one working file:

```
mkdir -p ~/projects/greeter
cd ~/projects/greeter
git init
```

Create `greet.py`:

```
def greet(name):
    return "Hello, " + name + "!"

print(greet("Ada"))
```

Check it runs, then commit:

```
python3 greet.py
git add greet.py
git commit -m "Add greeting"
```

Expected: Hello, Ada!, then a root-commit line.

Step 2: Try the Idea on a Branch

The idea: a friendlier greeting. Create a branch for it, so main stays exactly as it is:

```
git switch -c friendly
```

On this branch, change the return line to return "Hi there, " + name + "!", run the script, and commit:

```
python3 greet.py
git commit -am "Use a friendlier greeting"
```

Expected: Hi there, Ada!. Now switch back with `git switch main` and run `python3 greet.py` again. **Expected:** Hello, Ada!. The working version is untouched; Git swapped the file back when you switched.

Step 3: Meanwhile, main Moves On

Real projects don't stand still while you experiment. On main, tidy the same line into an f-string, return `f"Hello, {name}!"`, and commit:

```
git commit -am "Use an f-string for the greeting"
git log --oneline --graph --all

* 165c65b Use a friendlier greeting
| * c2ed7e4 Use an f-string for the greeting
|/
* 180f173 Add greeting
```

`--all` shows every branch. The history has forked: both branches changed the same line in different ways.

Step 4: Merge, and Meet a Conflict

The idea worked, so bring it into main:

```
git merge friendly

Auto-merging greet.py
CONFLICT (content): Merge conflict in greet.py
Automatic merge failed; fix conflicts and then commit the result.
```

Nothing is broken. Git has paused the merge because it can't guess which version of that line you want. `git status` says so, and lists the file under both `modified`. Open `greet.py`:

```
def greet(name):
<<<<<< HEAD
    return f"Hello, {name}!"
=====
    return "Hi there, " + name + "!"
>>>>>> friendly

print(greet("Ada"))
```

Git wrote both versions into the file between conflict markers. Everything between `<<<<<< HEAD` and `=====` is your current branch's version. Everything between `=====` and `>>>>>> friendly` is the incoming branch's version.

Step 5: Resolve It

Resolving means editing the file into what it should be, and deleting all three marker lines. You can keep one side, the other, or combine them. Here the best answer combines both ideas: the f-string and the friendlier words.

```
def greet(name):
    return f"Hi there, {name}!"

print(greet("Ada"))
```

Run it to make sure it works, then tell Git the conflict is resolved by staging the file, and finish the merge:

```
python3 greet.py
git add greet.py
git commit --no-edit
git log --oneline --graph

Hi there, Ada!
[main beb0c29] Merge branch 'friendly'
*   beb0c29 Merge branch 'friendly'
|\
| * 165c65b Use a friendlier greeting
* | c2ed7e4 Use an f-string for the greeting
|/
* 180f173 Add greeting
```

`--no-edit` accepts Git's ready-made message. The idea is now in `main`, so delete its branch with `git branch -d friendly`.



If a conflict gets confusing, `git merge --abort` puts everything back to how it was before the merge. Nothing is lost, and you can try again.

Step 6: Rebasing, in Brief

A merge keeps history exactly as it happened, forks and all. There's a second way to bring a branch up to date: **rebase**. It lifts your branch's commits off and replays them on top of the latest `main`, as if you'd started the branch just now.

Make a branch, commit on it, then let `main` move on:

```
git switch -c shout
echo 'print(greet("Ben").upper())' >> greet.py
git commit -am "Shout at Ben"
git switch main
```

On main, add the line `"""A tiny greeter."""` at the top of `greet.py` and commit it as “Add a docstring”. The history has forked again:

```
* 2ebbb10 Add a docstring
| * eeb1248 Shout at Ben
|/
* beb0c29 Merge branch 'friendly'
```

Now rebase shout onto main:

```
git switch shout
git rebase main
git log --oneline --graph --all -3
```

Successfully rebased and updated refs/heads/shout.

```
* 88709a8 Shout at Ben
* 2ebbb10 Add a docstring
* beb0c29 Merge branch 'friendly'
```

The fork is gone. “Shout at Ben” now sits on top of the docstring commit, with a new ID (88709a8 instead of eeb1248), because it’s technically a new commit. Merging it into main is now a simple fast-forward: `git switch main`, then `git merge shout.python3 greet.py` prints `Hi there, Ada!` and `HI THERE, BEN!`.



Rebase only commits that nobody else has. Rebasing rewrites commits into new ones, so if you rebase commits you’ve already pushed, everyone who pulled the old ones ends up with two conflicting histories. Rebase your own local work; merge shared work.

Step 7: Lose a Commit, Then Find It

Last, the rescue. Make an experiment and commit it:

```
git switch -c experiment
echo 'print(greet("World"))' >> greet.py
git commit -am "Greet the whole world"
git switch main
```

Now delete the branch. Git refuses with `-d`, because the commit isn't merged anywhere, which is a good warning. Force it with `-D`:

```
git branch -D experiment
```

```
Deleted branch experiment (was 207ad9c).
```

`git log --all` no longer shows “Greet the whole world”. It looks gone. But the `reflog` remembers every place HEAD has been:

```
git reflog -3
```

```
88709a8 HEAD@{0}: checkout: moving from experiment to main
```

```
207ad9c HEAD@{1}: commit: Greet the whole world
```

```
88709a8 HEAD@{2}: checkout: moving from main to experiment
```

There it is: 207ad9c, one step back, at HEAD@{1}. A branch is just a bookmark, so put a new bookmark on it:

```
git branch experiment 207ad9c
```

```
git log --oneline -1 experiment
```

```
207ad9c Greet the whole world
```

The commit is back. The same trick undoes an unwanted `git reset --hard HEAD~1`: find the old position in `git reflog`, then `git reset --hard HEAD@{1}`.



Git only keeps unreachable commits for a while (normally at least 30 days) before cleaning them up. The `reflog` is a safety net, not an archive. And it only knows about commits: uncommitted edits you throw away are gone for good.

Summary, Key Terms, and Review Questions

Summary

- A branch is a safe place to try an idea. Switching back to `main` restores the working version instantly.
- A merge conflict happens when both branches changed the same lines. Git pauses and writes both versions into the file between conflict markers.
- To resolve, edit the file into its final form, remove the markers, `git add` it, and commit. `git merge --abort` backs out.

- Rebase replays your commits on top of another branch, giving a straight history. Only rebase commits you haven't shared.
- Deleted branches and reset commits can be found in `git reflog` and brought back with a new branch or a reset.

Key Terms

Term	Meaning
Rebase	Replay a branch's commits on top of another commit, creating new commits

Review Questions

1. Why is the working version safe while you experiment on a branch?
2. In a conflict, which side sits between <<<<<< HEAD and =====?
3. What three things must you do to finish resolving a conflict?
4. How does history look after a rebase compared with a merge? Why do the rebased commits get new IDs?
5. You deleted a branch with `git branch -D` and regret it. Describe how to get it back.

You understand this when you can resolve a merge conflict without panic, and bring back a commit you thought was lost.

Lab: Work on a Project with Someone Else

In this lab you play two people, Ada and Ben, sharing one project through a remote. You'll see how Git refuses to let one person's push wipe out another's work, how `fetch` and `pull` bring changes together, and how teams use feature branches and pull requests to review work before it lands. You'll finish with forks, the open-source workflow, and licences.

The problem. Two of us edited the same file, and someone's work vanished.

The question. How do several people change one project at the same time without losing anyone's work?

You need no accounts. A bare repository on your own disk plays the part of GitHub, and two folders play Ada's and Ben's laptops. Paths in the output show `/home/ada` because one person runs the whole lab.

Step 1: Set Up the Shared Remote

Create the shared repository, then Ada's copy:

```
mkdir -p ~/collab && cd ~/collab
git init --bare ~/remotes/team.git
git clone ~/remotes/team.git ada
cd ada
git config user.name "Ada"
git config user.email "ada@example.com"
```

`git config` without `--global` sets the name for this repository only. That's how one computer can pretend to be two people. Git warns You appear to have cloned an empty repository, which is true and harmless.

Ada writes `shop.py`:

```
PRICES = {"apple": 0.5, "bread": 2.0}

def total(items):
    return sum(PRICES[item] for item in items)

print(total(["apple", "bread"]))
```

She commits and pushes:

```
git add shop.py
git commit -m "Add shop total"
git push
```

Expected: * [new branch] main -> main.

Now make Ben's copy the same way: from `~/collab`, run `git clone ~/remotes/team.git` ben, then set `user.name` to Ben and `user.email` to `ben@example.com` inside it. **Expected:** `git log --oneline` in ben shows Ada's commit.

Step 2: Both Work at Once

Ada adds milk to the prices in `shop.py`, commits, and pushes. It goes through. Meanwhile Ben, who hasn't seen Ada's change, adds a `README.md`, commits, and pushes:

```
git push

To /home/ada/remotes/team.git
! [rejected] main -> main (fetch first)
error: failed to push some refs to '/home/ada/remotes/team.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ...
```

This is the answer to the chapter's problem. When files are shared by email or a synced folder, the last person to save wins, and the other's work silently vanishes. Git refuses instead. Ben's push would have replaced Ada's commit, so the remote rejects it until Ben has Ada's work too.

Step 3: Fetch, Look, Then Pull

`git fetch` downloads new commits from the remote without touching your files or your branches. They land in a **remote-tracking branch**, `origin/main`: Git's record of where `main` was on the remote when you last checked.

```
git fetch
git status
git log --oneline --graph --all

On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
...
* 193c83f Add README
| * b2f7c97 Add milk to prices
|/
* 04da6ba Add shop total
```

History has forked, just like two branches. `git pull` is simply `git fetch` followed by a merge of `origin/main` into your branch. When the histories have diverged, Git wants you to choose once how pulls should combine them. Pick merging, which you already know:

```
git config --global pull.rebase false
git pull

Merge made by the 'ort' strategy.
shop.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Git opens your editor with a merge message; save and close it. Now Ben has both changes, and `git push` succeeds. Back in ada, a `git pull` fast-forwards and brings in Ben's README.

Expected: both folders contain README.md and the milk price, and `git log --oneline` shows the same commit at the top in both.

If Ada and Ben had changed the same line, `git pull` would have stopped with a conflict. You'd resolve it exactly as in Chapter 10: edit, `git add`, commit, then push.



Pull before you start work and before you push. The less time your copy spends out of date, the smaller any conflict will be.

Step 4: A Feature Branch

Pushing straight to `main` works for two people. Teams usually protect `main` and make every change on a **feature branch** first, so someone can review it before it lands. Ben wants to add discounts:

```
git switch -c add-discount
```

He appends a function to `shop.py`:

```
def with_discount(amount, percent):
    return round(amount * (1 - percent / 100), 2)
```

He commits it as “Add percentage discount” and pushes the branch, not `main`:

```
git push -u origin add-discount
```

```
To /home/ada/remotes/team.git
* [new branch]      add-discount -> add-discount
branch 'add-discount' set up to track 'origin/add-discount'.
```

Step 5: The Pull Request and the Review

On GitHub, this is where Ben opens a **pull request**: a web page that proposes “please merge `add-discount` into `main`”, with a place to discuss it. Pull requests are a feature of hosting sites, not of Git. Here’s what Ben would see:

1. A yellow banner on the repository page says his branch had recent pushes, with a *Compare & pull request* button.
2. He picks the target (`main`) and his branch, writes a title and a description of what changed and why, and clicks *Create pull request*.
3. The pull request page has tabs. *Conversation* holds the discussion. *Commits* lists his commits. *Files changed* shows the diff.

Ada does a code review: she reads the change and leaves comments on specific lines. She can then choose *Comment*, *Approve*, or *Request changes*. You can do the same review from the terminal:

```
cd ~/collab/ada
git fetch
git diff main...origin/add-discount

...
    print(total(["apple", "bread"]))
+
+
+def with_discount(amount, percent):
+    return round(amount * (1 - percent / 100), 2)
```

The three dots mean “what the branch changed since it split from `main`”. Ada asks for an example of the function in use. Ben adds `print(with_discount(10, 15))`, commits “Show a discount example”, and runs `git push` again. On GitHub, the new commit simply appears in the same pull request. There’s no need to open a new one.

Step 6: Merge and Clean Up

Satisfied, Ada merges. On GitHub she’d click *Merge pull request*, then *Delete branch*. Locally, the same thing is:

```
git merge --no-ff --no-edit origin/add-discount
git push
git push origin --delete add-discount
```

`--no-ff` makes a merge commit even though a fast-forward was possible, as GitHub’s button does, so the history records that a reviewed branch came in. Ben then updates and tidies his copy:

```
cd ~/collab/ben
git switch main
git pull
git branch -d add-discount
```

Expected: `python3 shop.py` prints 2.5 and 8.5 in both folders.

Step 7: Forks

You can't push to a stranger's project. So on GitHub you click **Fork**, which makes your own full copy of their repository under your account. You push to your fork, then open a pull request from your fork to theirs. The original project is called **upstream**.

Simulate it with a third person, Cy. A fork is just a bare clone:

```
cd ~/collab
git clone --bare ~/remotes/team.git ~/remotes/cy-fork.git
git clone ~/remotes/cy-fork.git cy
cd cy
git remote add upstream ~/remotes/team.git
git remote -v

origin    /home/ada/remotes/cy-fork.git (fetch)
origin    /home/ada/remotes/cy-fork.git (push)
upstream  /home/ada/remotes/team.git (fetch)
upstream  /home/ada/remotes/team.git (push)
```

Cy's origin is her fork, which she can push to. upstream is the original. When the original moves on, she keeps her fork current like this:

```
git fetch upstream
git merge upstream/main
git push origin main
```

Expected: after Ada pushes a new commit to `team.git`, these three commands bring it into Cy's copy and her fork.

The Open-Source Workflow

Open source means the code is public and anyone may use, study, change, and share it, under the terms of its licence. Contributing to a project usually goes like this:

1. Read the README and any CONTRIBUTING file. They explain how the project wants to work.
2. Find or open an issue, the project's to-do and bug list on GitHub. Say what you plan to do before writing code, so nobody's effort is wasted.
3. Fork, clone, create a feature branch, commit, and push to your fork.
4. Open a pull request to upstream, respond to review, and push fixes to the same branch.

A **licence** is the file, usually `LICENSE`, that says what others may do with the code. Three are common:

Licence	In short
MIT	Do almost anything; keep the copyright notice
Apache 2.0	Like MIT, plus explicit patent terms
GPL 3.0	You may share changes, but shared versions must stay GPL



Public code with no licence is not free to use. By default, copyright means all rights are reserved. Check the licence before you copy code into your project, and add one to anything you publish.

Summary, Key Terms, and Review Questions

Summary

- A shared remote is the meeting point. Git rejects a push that would overwrite someone else's commits.
- `git fetch` downloads commits into remote-tracking branches like `origin/main`; `git pull` fetches and merges.
- Teams make changes on feature branches, push them, and open pull requests for code review before merging.
- More commits pushed to the branch update the same pull request. After merging, delete the branch.
- A fork is your own copy of someone else's repository; `upstream` points at the original.
- Open-source work flows through issues, forks, and pull requests. The licence decides what you may do with the code.

Key Terms

Term	Meaning
Remote-tracking branch	Git's record of a remote branch, such as <code>origin/main</code>
Feature branch	A branch holding one change, to be reviewed and merged
Pull request	A hosting site's proposal and discussion page for merging a branch
Fork	Your own copy of someone else's repository on a hosting site
Upstream	The original repository a fork was copied from
Open source	Code anyone may use, change, and share under its licence
Licence	The terms that say what others may do with the code

Review Questions

1. Why did Git reject Ben's push, and what would have happened to Ada's work if it hadn't?
2. What's the difference between `git fetch` and `git pull`?
3. Why do teams use feature branches and pull requests instead of pushing to `main`?
4. How do you update an open pull request after a reviewer asks for changes?
5. In a fork, what do `origin` and `upstream` point to?
6. You find useful code on GitHub with no licence. May you copy it into your project?

You understand this when you can take a change from a feature branch, through review, into a shared `main`, without anyone's work being lost.

PART IV

What Next

Security, Speed, Failure, and Cost

This chapter looks back at the terminal and Git through four questions that every tool raises sooner or later: what can go wrong, what makes it slow, how it breaks, and what it costs. Each section is short and practical: the few habits that protect your work, your time, and your money.

The problem. The tools work for me now, but I don't know what could go badly wrong, or what it would cost me.

The question. What are the real risks of the terminal and Git, and which habits keep me safe?

Security: What Can Go Wrong, and What Limits the Damage

Secrets in your shell history. Your shell saves the commands you type, in `~/.bash_history` or `~/.zsh_history`. Type `export API_KEY=...` at the prompt and the key sits in that file for years. In bash, setting `HISTCONTROL=ignorespace` makes the shell skip any command that starts with a space:

```
export HISTCONTROL=ignorespace
export API_KEY=sk-live-4f9a2c
export COLOUR=blue
history 3
```

```
1 export HISTCONTROL=ignorespace
2 export COLOUR=blue
3 history 3
```

The line with the key never reached the history. (In zsh, the setting is `setopt HIST_IGNORE_SPACE`.) Better still, keep secrets in a `.env` file with `chmod 600` and never type them at all.

Secrets in Git. This is the big one. Once a secret is committed, deleting the file doesn't delete it from history. Here a project committed `.env`, then removed it and added it to `.gitignore`:

```
ls -a
git show HEAD~1:.env

.  .. .git .gitignore app.py
API_KEY=sk-live-4f9a2c
```

The file is gone from the folder, but one command pulls it straight out of the previous commit. Anyone who clones the repository can do the same, and bots scan public repositories for keys within minutes of a push.



If you ever commit a secret, treat it as stolen. Revoke or rotate it at the provider first (create a new key, disable the old one), then clean up the repository. Rewriting history with a tool such as `git filter-repo` doesn't help if someone has already copied the key.

The prevention is cheap: create `.gitignore` before your first commit, and always read `git status` and `git diff --staged` before you commit.

Permissions and privilege. The habits from Chapter 3 limit the damage when something does go wrong. Keep private files at 600, including SSH keys. Protect your private key with a passphrase. Don't work as root, and use `sudo` for single commands only. Read scripts before running them, especially anything piped from `curl` into a shell.

Speed: What Makes This Slow

Searching too much. Most slow commands are searching far more than you need. `grep -r` or `find` started at `/` or your home folder crawls through every file, including huge folders like `.venv`, `node_modules`, and `.git`. Start as close to the target as you can, and skip the heavy folders: `grep -r --exclude-dir=.venv TODO ..`

Large repositories. Git keeps every version of every file forever. That's wonderful for code and costly for big binary files, such as videos, datasets, and build outputs. Watch what happens when a 20 MB file is committed and then deleted:

```
du -sh .git

19M    .git
```

The file is gone from the project, but it's still in history, so every clone downloads it, for good. Before the video, `.git` was 128 KB. Three habits keep repositories fast:

- Put build outputs, virtual environments, and data files in `.gitignore`.
- Store large assets somewhere else, or use Git LFS (Large File Storage), a Git add-on that keeps big files outside the history.
- When you only need the latest version of someone’s huge project, clone just that: `git clone --depth 1 <url>`.

Waiting on the network. `git status`, `diff`, `log`, `commit`, and `branch` are fast because they never leave your machine. Only `clone`, `fetch`, `pull`, and `push` talk to the remote. If Git feels slow, check which kind of command you’re running.

Failure: How It Breaks, and How to Diagnose It

Most failures in this book announce themselves with a clear message. Read the message first, then run the one command that checks it:

You see	Likely cause	Check with
command not found	Typo, not installed, or not on PATH	<code>which name</code> , <code>echo \$PATH</code>
Permission denied	Not yours, not executable, or a closed folder	<code>ls -l</code> on the file and its folders
No space left on device	Disk full, often from logs or backups	<code>df -h</code> , then <code>du -sh *</code> in suspect folders
A background program vanished	It crashed or was killed	<code>Its log</code> , or <code>journalctl -u name</code>
A script “succeeded” but did damage	Errors were ignored	<code>Add set -euo pipefail</code>
! [rejected] ... (fetch first)	Someone pushed before you	<code>git pull</code> , then <code>push</code>
CONFLICT	Both sides changed the same lines	<code>git status</code> , then <code>resolve</code>
HEAD detached at ...	You checked out a commit, not a branch	<code>git switch main</code>
A commit seems lost	Branch deleted or reset	<code>git reflog</code>

Two failures can’t be fixed afterwards, so prevent them. First, uncommitted work that you discard with `git restore` or `git reset --hard` is gone, because Git never saw it; commit small and often. Second, a laptop is a single point of failure; push to a remote regularly, because a repository that exists in one place isn’t backed up.



When you’re stuck, run `git status`. It almost always says what state you’re in and prints the exact command to get out of it.

Cost: What This Costs You

The tools themselves are free. Linux, bash, zsh, the command-line tools, and Git are all open source, and they run on the laptop you already have. The real costs are elsewhere:

- *Time.* The first weeks in the terminal are slower than clicking. That cost is paid once, and the time it saves comes back every day after.
- *Hosting.* GitHub, GitLab, and Bitbucket all have free plans that cover public and private repositories for individuals and small teams. Paid plans add things like more automation minutes, more storage for large files, and organisation controls. Check current prices before you rely on a limit.
- *Disk and bandwidth.* Big files in history cost every person who clones the project, on every clone.
- *Mistakes.* The most expensive items in this chapter are the silent ones. A cloud key leaked in a public repository can be used to run up a large bill before you notice. A week of work that was never pushed dies with a stolen laptop. Both are prevented by habits that cost nothing: ignore secrets, commit often, push daily.

Where to Go from Here

You can now open a terminal and know what you're typing. You can find, read, and change files, keep programs running, write a script you trust, and use Git to save your work, try ideas safely, and share a project with other people. This last chapter suggests what to learn next, and just as usefully, what you can safely ignore for a while.

Make It a Habit First

The fastest way to get good at these tools is to use them for everything, starting today:

- Put every project in a Git repository from its first file, and push it to a remote.
- Do small jobs in the terminal even when a mouse would be quicker at first: moving files, searching logs, renaming things.
- When you type the same commands twice, make them a script, with `set -euo pipefail` and a pass through `shellcheck`.
- When a command surprises you, read its `man` page instead of searching for a fix to paste.

A month of this does more than any course.

What to Learn Next

More Git, as you need it. A few commands earn their place once the basics feel natural. `git stash` puts half-finished work aside so you can switch branches. `git tag` marks releases. `git rebase -i` (interactive rebase) lets you tidy your own commits before you open a pull request. `git bisect` finds the commit that introduced a bug by testing halfway points. Learn each one when a real problem calls for it.

Automated checks. Hosting sites can run your tests on every push and every pull request, and block merging when they fail. On GitHub this is GitHub Actions. It's the natural next step after pull requests, and it's covered in *Infrastructure Engineering from Zero*.

A terminal editor. Sooner or later you'll edit a file on a server with no graphical editor. Learn enough nano to edit and save, or enough Vim to edit, save, and quit (:wq).

What's under the shell. This book explained only as much about the kernel, processes, and memory as the terminal needs. *Operating Systems from Zero* goes underneath: system calls, how processes and threads really run, virtual memory, and file systems.

The network. SSH, git push, and curl all cross a network you've treated as a pipe. *Networking from Zero* explains what happens on the way: addresses, ports, DNS, TCP, and HTTP. It's the book to read when "it works on my machine" but nobody else can reach it.

Building with these tools. *Web Applications from Zero* and *APIs to FastAPI* put the terminal and Git to work on real programs, and *The Complete Software Engineering Guide* takes one project from an idea all the way to production, using everything here along the way.

What to Skip for Now

Plenty of material online is correct but not yet useful to you. It's safe to skip:

- *Git's internals.* How objects, trees, and packfiles are stored is fascinating, and you don't need it to use Git well. The same goes for "plumbing" commands like `git cat-file` and `git update-ref`.
- *Complex branching strategies.* Models with many long-lived branches (develop, release, hotfix) solve problems of large teams shipping versioned software. A main branch plus short feature branches serves almost everyone else.
- *Submodules and hooks.* Useful in specific situations; you'll know when you reach one.
- *Every option of every command.* Learn the handful you use, and look the rest up when you need them.
- *Advanced awk and sed programs.* Past a few lines, write the job in Python instead. It's easier to read and test.
- *Shell customisation rabbit holes.* Fancy prompts, plugin frameworks, and elaborate dotfiles are fun, but they don't make you more capable. A plain shell you understand is better than a clever one you don't.

One Last Thing

Every developer you admire once typed `ls` and wondered what it would do. The difference between them and a beginner isn't talent; it's hours in the terminal and a history of commits. Open a terminal, start a repository, and make your first commit today. Then make another one tomorrow.

Command Cheat Sheet

The commands from this book, grouped by job. Where Linux and macOS differ, both are shown.

Files, Folders, and Text

Command	What it does
<code>pwd, ls -la, cd dir</code>	Where am I; list everything; go into dir (<code>cd ..</code> up, <code>cd home</code>)
<code>mkdir -p a/b, touch f</code>	Make folders with parents; create an empty file
<code>cp a b, mv a b, rm f</code>	Copy, move or rename, remove (<code>-r</code> for folders; no undo)
<code>cat f, less f, head/tail -n 5 f</code>	Print, page through, first or last lines (<code>tail -f</code> follows)
<code>du -sh d, df -h</code>	Size of a folder; free space on each disk
<code>find . -name '*.py'</code>	Find files by name (also <code>-type</code> , <code>-mtime</code> , <code>-size</code>)
<code>grep -rn word .</code>	Search inside files, with line numbers (<code>-c</code> , <code>-i</code> , <code>-v</code> , <code>-E</code>)
<code>sed 's/a/b/g' f</code>	Print a changed copy; <code>-i</code> edits in place (macOS: <code>-i ''</code>)
<code>cut -d' ' -f5 sort uniq -c sort -rn</code>	Keep a column, then count and rank its values
<code>jq '.key' f.json</code>	Pull fields out of JSON; <code>jq .</code> pretty-prints

The Shell, Permissions, and Settings

Command	What it does
<code>cmd > f,>> f,2> f,a b</code>	Output to a file; append; errors; feed a into b
<code>echo \$PATH,which cmd</code>	Where commands are looked up; which one runs
<code>export NAME=value</code>	Pass a setting to programs (keep it in <code>~/ .bashrc</code>)
<code>id,ls -l f</code>	Who you are; a file's owner and permissions
<code>chmod u+x f,chmod 600 f</code>	Let the owner run it; make it private (644, 755)
<code>sudo cmd</code>	Run one command as root

Processes, Software, and Remote Work

Command	What it does
<code>ps aux,pgrep -af name</code>	List processes; find one by name (macOS: <code>-fl</code>)
<code>cmd &,nohup cmd > log 2>&1 &</code>	Run in the background; outlive the terminal
<code>kill PID,kill -9 PID</code>	Ask to stop (SIGTERM); force stop (SIGKILL)
<code>crontab -e</code>	Schedule a job
<code>sudo apt install x,brew install x</code>	Install software (Debian/Ubuntu; macOS)
<code>python3 -m venv .venv</code>	Create a virtual environment
<code>ssh-keygen -t ed25519,ssh user@host</code>	Make a key pair; open a shell on another machine
<code>rsync -av src/ dst/,tmux new -s x</code>	Copy only what changed; a lasting session

Scripts

Syntax	What it does
<code>#!/usr/bin/env bash,set -euo pipefail</code>	Run with bash; stop at the first error
<code>"\$name",\$(cmd),\${1:-default}</code>	A quoted variable; a command's output; an argument
<code>if [[-f "\$f"]],for f in *.md</code>	Test and branch; repeat for each item
<code>exit 1,>&2,shellcheck s.sh</code>	Fail with a code; write to stderr; check for bugs

Git Cheat Sheet

The Git commands from this book, in the order you'll usually need them.

Set Up Once

Command	What it does
<code>git config --global user.name "Ada Lovelace"</code>	Name recorded in your commits (and <code>user.email</code>)
<code>git config --global init.defaultBranch main</code>	New repositories start on <code>main</code>
<code>git config --global pull.rebase false</code>	Pulls combine diverged work by merging

Every Day

Command	What it does
<code>git init, git clone <url-or-path></code>	Start a repository; copy an existing one
<code>git status</code>	What changed, and what's staged
<code>git diff, git diff --staged</code>	Unstaged changes; what the next commit will hold
<code>git add file, git commit -m "Message"</code>	Stage a change; record the snapshot (-am does both)
<code>git log --oneline --graph --all</code>	History of every branch, drawn as a graph
<code>git switch -c name, git switch name</code>	Create and switch to a branch; switch to one
<code>git merge name, git merge --abort</code>	Merge name into this branch; back out of a conflict
<code>git rebase main</code>	Replay this branch on top of main (unshared commits only)
<code>git branch -d name</code>	Delete a merged branch (-D forces it)

Sharing

Command	What it does
<code>git init --bare name.git</code>	Make a repository to use as a shared remote
<code>git remote add origin <url-or-path></code>	Name a remote (<code>git remote -v</code> lists them)
<code>git push -u origin main</code>	First push of a branch, remembering the pairing
<code>git push, git pull, git fetch</code>	Send; fetch and merge; fetch only
<code>git diff main...origin/branch</code>	What a branch changed since it split from main
<code>git push origin --delete name</code>	Delete a branch on the remote

Undo

Situation	Command
Discard unstaged edits to a file	<code>git restore file</code>
Unstage a file, keeping the edits	<code>git restore --staged file</code>
Fix the last commit (message or files)	<code>git commit --amend</code>
Undo a pushed commit safely	<code>git revert <id></code>
Undo the last local commit	<code>git reset --soft HEAD~1</code> (keep changes) or <code>--hard</code> (discard)
Find a “lost” commit and bring it back	<code>git reflog</code> , then <code>git branch name <id></code>

Resolving a Conflict

1. `git status` lists files marked both modified.
2. Edit each file to its final form and delete the <<<<<<, =====, and >>>>>> lines.
3. Run the code to check it works, then `git add` each file and `git commit`.

Glossary

Term	Meaning
Argument	What a command acts on; in a script, read as \$1, \$2, ...
Bare repository	A repository with history but no working tree, used as a shared remote
Branch	A movable name that points at one commit
Clone	A full copy of an existing repository, history included
Commit	A snapshot with an author, time, message, and parent
cron	The classic Unix scheduler that runs commands at set times
Distribution	The Linux kernel bundled with tools, such as Ubuntu
Environment variable	A named setting, such as HOME, that programs inherit
Exit code	The number a command returns: 0 for success
Fast-forward	A merge that only moves the branch pointer forward
Feature branch	A branch that holds one change, to be reviewed and merged
Fork	Your own copy of someone else's repository on a hosting site
Git	The free version control program that runs on your computer
GitHub	A website that hosts Git repositories, with pull requests
.gitignore	A file listing patterns of files Git should not track
HEAD	Git's marker for "you are here": normally the current branch
Hidden file	A file whose name starts with .; plain ls doesn't show it
Home directory	Your personal folder, written ~
Idempotent	Safe to run many times, always leaving the same result
Init system	PID 1, which starts and supervises services
JSON	A text format for data made of objects ({}) and lists ([])
Kernel	The core of the operating system, which controls access
Key pair	A linked public and private key that prove who you are
Least privilege	Giving each user and program only the access it needs, and no more

Term	Meaning
Licence	The terms that say what others may do with the code
Linux	A Unix-like kernel, and by extension the operating systems built on it
Log	A record of timestamped messages written by a program
Man page	The built-in manual for a command, opened with <code>man</code>
Merge	Combining the work of two branches
Merge conflict	When both branches changed the same lines, so Git needs you to choose
Open source	Code that anyone may use, study, change, and share under its licence
Operating system	The software that shares one computer among many programs
Option	A flag, such as <code>-l</code> , that changes how a command behaves
Package manager	A tool that installs and updates software, such as <code>apt</code>
PATH	The folders the shell searches for commands
Path	The address of a file or folder, such as <code>/home/ada/notes.txt</code>
Permissions	The read, write, and execute rules for a file's owner, group, and others
PID	Process ID: the number the kernel gives each process
Pipe	The <code>'</code>
Private key	The secret half of a key pair; it never leaves your machine
Process	A running instance of a program
Public key	The half of a key pair you share, installed on servers like a padlock
Pull	Bringing a remote's new commits into your repository
Pull request	A hosting site's page proposing and reviewing a merge
Push	Sending your new commits to a remote
Rebase	Replaying a branch's commits on top of another
Redirection	Sending a program's output to a file with <code>></code> , <code>>></code> , or <code>2></code>
Reflog	Git's record of everywhere HEAD has pointed, used to find lost commits
Regular expression	A small pattern language for matching text
Remote	Another copy of a repository that you push to and pull from
Remote-tracking branch	Git's record of a remote's branch, such as <code>origin/main</code>
Repository	A project's collected history, stored in the <code>.git</code> folder
root	The superuser (UID 0), who bypasses permission checks
Script	A file of commands that the shell runs in order
Service	A background program supervised by the system; also called a daemon
Shebang	The <code>#!</code> first line of a script, naming the program that should run it

Term	Meaning
Shell	The program that reads your commands and runs them, such as <code>bash</code> or <code>zsh</code>
Signal	A small numbered message sent to a process, such as <code>SIGTERM</code> or <code>SIGKILL</code>
SSH	Secure Shell: an encrypted way to use a shell on another machine
Staging area	The changes chosen for the next commit; also called the index
Standard error	The output stream a program uses for error messages (<code>stderr</code>)
Standard input	The stream a program reads from, usually the keyboard (<code>stdin</code>)
Standard output	The stream a program writes results to, usually the screen (<code>stdout</code>)
Strict mode	<code>set -euo pipefail</code> : stop a script at the first error
<code>sudo</code>	Runs one command as root, after checking your password
System call	A program's request to the kernel, such as "open this file"
Terminal	The window that shows text and passes your typing to the shell
Unix	The older operating system whose design Linux and macOS follow
Upstream	The original repository that a fork was copied from
Variable	A name that holds a value; in the shell, read with <code>\$name</code>
Version control	A system that records every state of a project
Virtual environment	A per-project folder with its own Python interpreter and packages
Working directory	The folder the shell is currently in
Working tree	The project files you see and edit, outside <code>.git</code>
WSL	Windows Subsystem for Linux: a real Linux running inside Windows

Index

Symbols

.gitignore, 50

A

argument, 8, 61

B

bare repository, 52

branch, 22

C

clone, 23

commit, 21

cron, 39

D

distribution, 3

E

environment variables, 33

exit code, 59

F

fast-forward, 51

feature branch, 76

fork, 78

G

Git, 24

GitHub, 24

H

HEAD, 22

hidden file, 15

home directory, 8

I

idempotent, 60

init system, 37

J

JSON, 32

K

kernel, 2

key pair, 42

L

least privilege, 18

licence, 79

Linux, 3

log, 38

M

man page, 11

merge, 23

merge conflict, 23

O

open source, 78

operating system, 2

option, 8

P

- package manager, 41
- path, 8
- PATH, 9
- permissions, 16
- PID, 35
- pipe, 11
- private key, 42
- process, 35
- public key, 42
- pull, 23
- pull request, 76
- push, 23

R

- rebase, 69
- redirection, 10
- reflog, 53
- regular expression, 30
- remote, 23
- remote-tracking branch, 75
- repository, 21
- root, 16

S

- script, 58
- service, 37

- shebang, 58

- shell, 2

- signal, 36

- SSH, 42

- staging area, 47

- standard error, 10

- standard input, 10

- standard output, 10

- strict mode, 59

- sudo, 18

- system call, 3

T

- terminal, 2

U

- Unix, 4

- upstream, 78

V

- variable, 60

- version control, 21

- virtual environment, 42

W

- working directory, 8

- working tree, 47

- WSL, 4

Linux and Git from Zero

Every tutorial says “open the terminal”, and when your code breaks you can’t get the old version back. This book starts from zero and makes the two tools every developer uses every day feel ordinary: the Linux command line, and Git.

You’ll learn how to:

- Move around, find things, and read logs from the terminal
- Understand files, permissions, processes, and services
- Save every change with Git, and undo mistakes safely
- Try ideas on branches and work on a project with others
- Turn daily chores into small, safe shell scripts